La computadora como laboratorio filosófico: experimentos y herramientas para el modelado de sistemas reflexivos

por
Rocío Perez Sbarato

Presentado ante la FACULTAD DE MATEMÁTICA, ASTRONOMÍA, FÍSICA Y COMPUTACIÓN como parte de los requerimientos para la obtención del grado de Licenciado en Ciencias de la Computación de la

UNIVERSIDAD NACIONAL DE CÓRDOBA

Marzo, 2026

Director:
Javier Blanco

Este trabajo se distribuye bajo una licencia Creative Commons Atribución 4.0 Internacional

A mis profesoras y profesores,
por inspirarme a soñar.

A mis compañeras y compañeros,
por soñar conmigo.

Agradecimientos

Primero que todo, gracias a mi familia por mantenerme durante estos seis años e inculcarme los valores de la autenticidad, la integridad y el amor por el conocimiento. Gracias a Benja, por apoyarme y acompañarme durante la incertidumbre de la vida adulta. Un millón de gracias a los/as compañeros/as de mi camada, por encontrar el humor en nuestro trayecto compartido por parciales, exámenes y, ahora, la vida profesional. Gracias a los animales que me acompañaron en este trayecto: Blue, Blacky, Thorfinn, Brisa y Rosquetita. Gracias a la Universidad Nacional de Córdoba por las conexiones hermosas y oportunidades increíbles que me ha brindado. Agradezco especialmente a Javier, cuya guía y confianza fueron cruciales para mi crecimiento profesional y personal.

Resumen

La reflexividad está presente en campos tan diversos como la biología, la lingüística y las ciencias cognitivas, donde desempeña un papel central en sistemas de conocimiento complejos. Tomando a las paradojas como casos paradigmáticos de sistemas reflexivos, el objetivo principal de esta investigación es emplear la computadora como una herramienta activa para explorar y esclarecer su estructura referencial.

En primer lugar, se analizan los antecedentes en el estudio de la composición de las paradojas. En segundo lugar, se programa en Haskell la librería Hypersets a partir del modelado lógico de la teoría de conjuntos no bien fundados. En tercer lugar, se utiliza esta herramienta para visualizar la estructura referencial de un grupo representativo de las paradojas, resaltando la circularidad como rasgo central. Finalmente, se discuten posibles extensiones y limitaciones de los modelos propuestos, proyectando un marco operativo prometedor para el análisis de fenómenos reflexivos.

Palabras clave: hypersets, modelo lógico, programación funcional, paradojas

Abstract

Reflexivity is present in diverse fields such as biology, linguistics, and the cognitive sciences, where it plays a central role in complex knowledge systems. The principal goal of this investigation is to employ the computer as an active tool for exploring and elucidating the referential structure of paradoxes, understood as paradigmatic cases of reflexive systems.

Firstly, the relevant literature on the compositional study of paradoxes is reviewed. Secondly, the Hypersets library is implemented in Haskell based on the logical modeling of non-well-founded set theory. Thirdly, this tool is used to visualize the referential structure of a representative group of paradoxes, highlighting circularity as a central trait. Finally, the limitations and possible extensions of the proposed models are discussed, outlining a promising operational framework for the analysis of reflexive phenomena.

Keywords— hypersets, logical modeling, functional programming, paradox

Índice

Agradecimientos **II**

Resumen **III**

Abstract **III**

1. Introducción **1**

2. Antecedentes y estado del arte **4**
2.1. Sistemas reflexivos y paradojas 4
2.2. La estructura de las paradojas autorreferenciales 7
2.2.1. Aczel y Barwise: teoría de *hypersets* y verdad 7
2.2.2. Priest: paradojas autorreferenciales del *Inclosure Scheme* . 9
2.2.3. Grim y Rescher: diagramas estructurales de la oscilación paradójica 13
2.3. El modelado computacional de problemas filosóficos 19
2.3.1. La programación como exploración 19
2.3.2. El horizonte interdisciplinario y pedagógico del modelado filosófico 22
2.4. Próximos pasos en el laboratorio filosófico 23

3. Librería Hypersets **24**
3.1. De conjunto a grafo 24
3.2. De ecuaciones a conjunto 33
3.3. De ecuaciones a grafos 36
3.4. Limitaciones y observaciones 40

4. Modelos y simulaciones **41**
4.1. Proposiciones sin autorreferencia y sus paradojas 42
4.2. Proposiciones autorreferenciales y sus paradojas 44
4.3. Contradicción en el *Inclosure Scheme* 47

5. Análisis de resultados **50**

6. Conclusión **52**

A. Repositorio de código **55**

Bibliografía **56**

1. Introducción

En el presente trabajo abordaremos el modelado lógico de los conceptos de referencia, reflexión y autorreferencialidad[1]. En diversos ámbitos – como la biología, la matemática, la epistemología, la filosofía del lenguaje y la inteligencia artificial – estos conceptos juegan un papel importante (Bartlett, 1992, pp. 5-24). Son herramientas argumentativas que operan dentro de sistemas de conocimiento y, a la vez, mensajeros de los límites aún inexplorados de sus dinámicas internas. También, funcionan como una llave: su análisis tiene el potencial de adquirir una comprensión más profunda de nuestras intenciones, a menudo inconscientes, al recurrir a la reflexividad (Bartlett & Suber, 1987, p. 17).

Dentro del amplio rango de fenómenos reflexivos, nos centramos en los más concretos y problemáticos: las paradojas lógicas, con y sin autorreferencia. Con autorreferencia nos referimos a la capacidad de un elemento para referirse a sí mismo. Para el lector que se encuentre en territorio desconocido, sospechamos que ha escuchado hablar de la paradoja del Barbero. Esta paradoja de autorreferencia consiste en el siguiente planteo:

Paradoja del Barbero

> En un pueblo en donde todo el mundo tiene que afeitarse, existe una ley que establece que "Solo a quienes no se afeiten a sí mismos les afeitará el barbero". Surge una pregunta: ¿El barbero se afeita a sí mismo? Si se afeita, entonces está siendo afeitado por la persona que solo afeita a quienes no se afeitan a sí mismos. Si no se afeita a sí mismo, entonces debe ir al barbero. Pero aparece el mismo problema, ya que sería afeitado por él mismo. Cada camino concluye en la misma contradicción.

Aunque pueda sonar como un acertijo absurdo, la paradoja del Barbero es una de muchas que reflejan la comprensión incompleta de nociones centrales como verdad o conjunto. Tal como describen Jon Barwise y John Etchemendy,

> "(...) El significado de una paradoja nunca es la paradoja en sí misma, sino aquello de lo que es un síntoma. Pues una paradoja muestra que nuestra comprensión de algún concepto básico (...) es crucialmente defectuosa (...) Y aunque esos casos límite puedan parecernos extraños o improbables, o incluso divertidos (...) Si los conceptos son importantes, esto no es motivo de risa (...)." (Barwise & Etchemendy, 1987, p. 4)

[1] Usamos reflexividad y autorreferencia de manera intercambiable.

El estudio de las paradojas autorreferenciales comenzó a tomarse en serio cuando, mediante argumentos reflexivos, se demostraron limitaciones en los sistemas formales. Por ejemplo, la demostración del teorema de Incompletitud de Gödel y del *Halting Problem*, que usaron una estructura fuertemente inspirada en el Mentiroso para su prueba formal (Bolander et al., 2006, pp. 6-7; Grim & Rescher, 2012, cap. 3). A lo largo de la historia de la lógica, se han propuesto diversos enfoques orientados a flexibilizar o restringir algunas de las características comúnmente asociadas a las paradojas.

Por su parte, Bertrand Russell, Alfred Tarski y Saul Kripke intentaron remover la reflexividad mediante la división, implícita y explícita, de sentencias en niveles (Bolander et al., 2006, pp. 8-13). De manera análoga, la teoría de conjuntos de Zermelo-Fraenkel, en su rol fundacional dentro de la matemática, prohíbe la autorreferencia a través de una jerarquía acumulativa de conjuntos (Bolander et al., 2006, p. 24).

En contraste con estas estrategias restrictivas, otros enfoques optan por no flexibilizar el estatus de un fenómeno paradójico. En esta línea, Graham Priest propone un cambio de paradigma lógico que admite sentencias simultáneamente verdaderas y falsas (Bolander, 2002, p. 24). Asimismo, se han desarrollado extensiones de la lógica que incorporan contexto, como la semántica situacional de Barwise y Etchemendy (Bolander, 2002, p. 25).

Aún así, estos enfoques no suelen ser considerados soluciones definitivas a las paradojas, ya que la introducción de tales restricciones falla en considerar casos límite o la posible formulación de paradojas reforzadas que vuelven a poner en tensión a las teorías propuestas. Al mismo tiempo, no por eso vale menos el estudio de las paradojas en sí mismas, como casos de estudio de fenómenos reflexivos que se manifiestan en una amplia variedad de disciplinas. Sin ir más lejos, Patrick Grim y Nicholas Rescher señalan que "(...) la reflexividad en el autoconocimiento y el autoconcepto es un aspecto crítico del tipo de seres que somos" (Grim & Rescher, 2012, p. 175).

Consideramos que explorar la reflexión por sí misma, tratándola como un fenómeno dinámico y operacional, puede aportar a una comprensión más profunda. Nuevas soluciones podrían salir de modelos novedosos de paradojas lógicas y su estructura referencial (Bolander, 2002, p. 25). De hecho, mientras que la relación entre el objeto referido y el objeto que refiere no sea plenamente comprendida, difícilmente se logre una comprensión completa de las paradojas de autorreferencia y de cualquier tipo (Bolander, 2002, p. 25). Por tanto, nuestra búsqueda consiste en el modelado de la estructura de las paradojas mediante su formalización como estructuras reflexivas con el fin de identificar características conceptuales comunes.

En el marco del modelado lógico de sistemas reflexivos, desarrollaremos una interpretación operacional de una teoría lógica que resulta especialmente opor-

tuna para nuestros objetivos exploratorios. En concreto, construimos la librería Hypersets: una herramienta que nos permite modelar la teoría de conjuntos no bien fundados de Peter Aczel. Tal teoría se basa en la ruptura de la jerarquía de conjuntos, lo que brinda comodidades a la hora de estudiar, simular y visualizar la reflexividad y circularidad (Aczel, 1988; Barwise & Etchemendy, 1987; Barwise & Moss, 1996). De este modo, la computadora funciona como laboratorio conceptual para explorar la estructura de paradojas. En otras palabras, la programación como método y resultado ocupa un lugar central en la exploración continua e investigación activa de la composición de las paradojas.

Bajo estas convicciones y camino metodológico propuesto, concebimos como hipótesis de este trabajo especial que los modelos lógicos generados representarán un resultado alentador para futuras investigaciones sobre el modelado de la estructura dinámica de sistemas reflexivos. Igualmente, la herramienta de simulación y visualización de paradojas mediante *hypersets* podría ser utilizada de manera didáctica para abordar dilemas de Filosofía, Ciencias de la Computación y Lógica, facilitando este vínculo intedisciplinario que beneficia a ambas áreas.

En este sentido, la investigación aborda un vacío presente en los trabajos previos. Notar que la mayoría de los antecedentes teóricos no contaban con la programación explícita de sus modelos. De entre los que contaban con un entorno computacional, estos no usaron la teoría de conjuntos no bien fundados. Además, si bien existen implementaciones de la teoría de *hypersets*, estas no habían sido aplicadas al modelado de paradojas. A su vez, los modelos formales de paradojas basados en *hypersets* no habían sido simulados mediante computadora ni desarrollados con un nivel de detalle centrado en la estructura interna de la paradoja, sino más bien orientados al análisis de las consecuencias de cada proposición y a la búsqueda de una solución.

El trabajo seguirá el siguiente orden. En el capítulo 2, presentaremos el marco conceptual y metodológico del trabajo. Expondremos antecedentes bibliográficos de distintos enfoques y herramientas para la caracterización de paradojas (Barendregt, 2003; Barwise & Etchemendy, 1987; Barwise & Moss, 1996; Bolander, 2002; Grim & Rescher, 2012; Priest, 1995). A su vez, destacaremos el rol de la computadora en la simulación de problemas filosóficos (Grim et al., 1998; Lim, 2023).

Continuando esta línea argumentativa, dedicaremos el capítulo 3 a la introducción de conceptos de la teoría de conjuntos no bien fundados conjuntamente a su implementación en Haskell. Las ventajas de utilizar este lenguaje funcional para la implementación de modelos lógicos son la posibilidad de usar tipos de datos a modo de sintáxis, funciones a modo de semántica y la comodidad para simular la recursión que trae consigo un entorno formal. El pipeline generado sistematiza la equivalencia de sistemas de ecuaciones y grafos de herencia mediante técnicas de *deep* y *shallow embedding*.

En el capítulo 4, exploraremos las paradojas mediante modelos lógicos de la teoría de *hypersets*. Construimos tipos de datos especificos para visualizar la estructura de las paradojas con o sin autorreferencia.

Finalmente, pasaremos en limpio las conclusiones respecto a los resultados alcanzados. Analizaremos los patrones establecidos y haremos nuevas preguntas para próximas investigaciones.

2. Antecedentes y estado del arte

Esta sección contextualiza el trabajo dentro del panorama histórico y conceptual de los sistemas reflexivos. En la sección 2.1, se definen los conceptos centrales de auto-referencia, paradoja y reflexión. Las secciones 2.2 y 2.3 recorren los antecedentes del trabajo, tanto en sus matices de modelado experimental como en la formalización de sistemas reflexivos. Esta primera parte de la investigación concluye destacando el potencial de un enfoque exploratorio, apoyado en modelos computacionales, para el estudio de los fenómenos reflexivos.

2.1. Sistemas reflexivos y paradojas

Al referirnos a algo estamos realizando la designación de un objeto a través de la palabra. Retomando la definición de Thomas Bolander, la referencia es una relación R entre una clase objetos referentes y una clase de objetos referidos (Bolander, 2002, p. 3).

Definición 2.1 (Referencia)**.** Una relación $(a,b) \in R$ significa que b es referido por a. El dominio de R es el conjunto de objetos que refieren. El rango de R es el conjunto de objetos que son referidos.

Referencia

Las siguientes frases son aplicaciones del concepto:

1. ‘Luz azul’ es un palíndromo
2. La única regla es que no hay reglas
3. La frase que usted está leyendo es la que estoy escribiendo
4. Esta oración tiene cuatro palabras
5. El menor entero positivo que no se puede describir con menos de quince palabras[2]

[2]Esta oración es la paradoja de Berry. Es contradictoria porque describe a dicho número en 14 palabras.

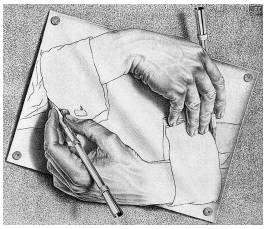


Figura 1: *Dibujando manos* (Escher, 1948)

En su presencia cotidiana y en diversas disciplinas, la referencia genera estructuras contradictorias y sistemas reflexivos. En el título de esta tesis, cuando hablamos de sistema reflexivo nos referimos a sistemas que exhiben reflexión. La reflexión es un caso especial de la referencia. Se utiliza para denotar cualquier situación en la que alguien o algo se ve a sí mismo "desde fuera" (Bolander, 2002). En la Figura 1 vemos una representación de reflexión circular.

Definición 2.2 (Reflexión)**.** Una relación de referencia R tiene reflexión si el dominio de objetos referentes es un subconjunto del rango de objetos referidos. Concretamente, $dom(R) \subseteq ran(R)$. Esto significa que todo objeto que refiere es también un objeto que es referido por algún otro elemento.

Reflexión

Henk Barendregt, en sus investigaciones sobre la reflexión, plantea que tiene tres elementos clave (Barendregt, 2003, pp. 1-3): un dominio de objetos significativos, una (de)codificación y la interacción. Para cada objeto, existe otro objeto – el código – a partir del cual el original puede ser reconstruido exactamente – es decir, decodificado. Respecto a la interacción, es un proceso de retroalimentación global que consiste en el encuentro entre los objetos y sus códigos. En la frase 1 la interacción se da entre la cita 'Luz azul', como código, y con las letras del abecedario, como objeto del dominio de referencia.

Autorreferencia

Contenida en la reflexión y un concepto central para este trabajo, la autorreferencia se refiere al caso particular de la referencia donde un objeto se refiere a sí mismo. La autorreferencia – directa o indirecta – se produce cuando un objeto a está contenido en un ciclo, en el grafo de la relación R, refiriéndose a sí mismo. La relación entre reflexión y referencia es de inclusión conceptual, donde la reflexión, formalizada a través de la relación de referencia, crea las condiciones en las que puede surgir la autorreferencia. Todas las oraciones salvo 1 y 5 tienen autorreferencia.

Un ejemplo visual y representativo de la auto-referencia se encuentra en la Figura 2. Un espectador observa un cuadro que alberga a la misma galería de arte en la cual se encuentra; el límite entre galería y cuadro se difumina porque se contienen mutuamente. Esta imagen nos confronta con la pregunta por las múltiples capas de referencia que pueden coexistir en una sola representación. En *Gödel,*

Escher, Bach, Douglas Hofstadter define esta sensación como el encuentro con un ciclo extraño; esto es, un aparente camino de salida que realiza un giro inesperado hacia el mismo punto de partida. El cruce entre jerarquías que observamos en la obra refleja un fenómeno más amplio, central en nuestra existencia y presente en diversos campos del conocimiento. Sobre la conciencia, la tesis más fuerte de Hofstadter es que los ciclos extraños están en el centro de nuestro entendimiento, a través del cual alcanzaremos una comprensión de la jerarquía enredada de niveles dentro de nuestra mente (Hofstadter, 1999, p. 710).

Si algo podemos sacar de estas definiciones y ejemplos, es que el estudio de la reflexión conlleva preguntarse cómo nos relacionamos con el mundo. Los círculos viciosos de esta naturaleza se presentan en la configuración tridimensional de las proteínas, en programas que se ejecutan a sí mismos y en compiladores que procesan el mismo lenguaje en que están implementados, en diversos casos de la lingüística y lógica, así como en demostraciones matemáticas con trucos autorreferenciales (Barendregt, 2003). En el contexto lingüístico, la frase 5 es lo que se denomina paradoja: una conclusión inconsistente partiendo de premisas aparentemente verdaderas. Además, una de de las paradojas semánticas más antiguas es la del Mentiroso, expresada en la afirmación "Esta oración es falsa". Si es verdadera, entonces es falsa. Pero si es falsa, entonces es verdadera.

Paradoja del Mentiroso

Con una estructura bastante similar, tenemos la paradoja de Russell en el área matemática. Entre los trabajos pioneros de la época, George Cantor se embarcó en el estudio del infinito y la construcción de la teoría de conjuntos fue el camino natural para hacerlo. Para ser aceptada por la comunidad matemática, la teoría de conjuntos debía estar construida bajo fundamentos lógicos y consistentes. Sin embargo, Bertrand Russell pudo encontrar una paradoja en la naciente teoría utilizando la técnica de diagonalización del mismo Cantor. Dado el conjunto russelliano R que está conformado por los conjuntos que no se contienen a sí mismos, la paradoja de Russell recae en la pregunta ¿$R \in R$?.

Paradoja de Russell

Bajo esta y una serie de contradicciones halladas en la estructura de la teoría de conjuntos, comenzó una batalla decisiva para las matemáticas del siglo XX (Ver Davis, 2018). Una paradoja con la estructura de Russell, además de la ya mencionada paradoja del Barbero, es la paradoja de Grelling. Consiste en el siguiente planteo: ¿Es la palabra "heterológico", que significa "no aplicable a sí mismo", un término heterológico?[3] En la siguiente sección, se profundizara sobre las familias de paradojas autorreferenciales y las estructuras que las subyacen. Por el momento, notar que las Figuras 1 y 2 crean un ciclo extraño que rompen la lógica convencional.

Paradoja de Heterological

Existen paradojas sin autorreferencia involucrada. La Paradoja de Yablo ilustra que una paradoja puede obtenerse al considerar una secuencia infinita de oraciones

Paradoja de Yablo

[3] "heterológico" es una traducción directa del término inglés "heterological".

Figura 2: *Galería de Litografías* (Escher, 1956)

donde cada oración se refiere a todas las oraciones subsiguientes.

$$S_i = \text{las oraciones } S_j \text{ con } j > i \text{ son falsas}, \quad \forall i \in \mathbb{N}$$

Ahora, supongamos que S_i es verdadera para algún i. Entonces, toda S_j con $j > i$ debe ser falsa. En particular, S_{i+1} debe ser falsa. Sin embargo, dado que S_{i+1} indica que todas las oraciones que le siguen son falsas, se sigue que S_{i+1} debe ser verdadera. Esto constituye una contradicción. Si suponemos que S_i es falsa para algún i, entonces al menos alguna de las oraciones que le siguen debe ser verdadera. Lo cual conduce nuevamente a una contradicción (Bolander, 2002, p. 13). Así, podemos ver que la circularidad no es necesaria para generar paradojas. La clave es una estructura no bien fundada, en este caso plasmada en una cadena infinita de referencia.

2.2. La estructura de las paradojas autorreferenciales

2.2.1. Aczel y Barwise: teoría de *hypersets* y verdad

En su búsqueda de evitar las paradojas de autorreferencia en la naciente teoría de conjuntos, Russell propuso el Principio de los Círculos Viciosos (PCV): "Cualquier cosa que involucre a toda una colección no debe ser uno de los elementos de esa colección"(Grim & Rescher, 2012, p. 14). Entre otros ejemplos, esto bloquea la paradoja de Russell y estructuras autorreferenciales como $X = \{X\}$. Bajo este principio, el Axioma de Fundación[4] (**FA**, por sus siglas en inglés) es incorporado a la teoría de conjuntos Zermelo–Fraenkel con el Axioma de Elección (**ZFC**). A

[4]Para todo conjunto no vacío A, se cumple que contiene un elemento disjunto de A. Formalmente, $\forall x((x \neq \emptyset) \rightarrow (\exists y)(y \cap x = \emptyset))$ (Barwise & Etchemendy, 1987, p. 34).

primera vista, la teoría de conjuntos clásica no parece ser óptima para modelar estructuras circulares de ningún tipo.

Los conjuntos que cumplen con **FA** son bien fundados. Llamamos bien fundado a un conjunto si la relación de pertenencia restringida a la clasura transitiva del conjunto es bien fundada, o sea que la relación de pertenencia dentro de los elementos del conjunto no forma ciclos o secuencias de descendencia infinitas de la forma $b_{n+1}Rb_n$, para cualquier n en los naturales (Barwise & Moss, 1996, p. 24). En otras palabras, en un conjunto bien fundado no existe una relación de pertenencia circular entre sus miembros.

Conjunto bien fundado

En teoría de conjuntos, la forma más natural de modelar una proposición p que se refiere a un objeto q es mediante un conjunto p que contiene al representante de q dentro de su relación de pertenencia hereditaria. Si una proposición es circular entonces el conjunto que la represente debe contenerse a sí mismo. Pero la existencia de un conjunto así no puede existir bajo el PCV y la teoría **ZFA** que solo aceptan conjuntos bien fundados. El Axioma de Fundación tiene como consecuencia la prohibición de conjuntos que se contienen a sí mismos, otros tipos de circularidad indirecta y cualquier estructura no bien fundada. Por un largo tiempo, al asociarse la circularidad con contradicción e inconsistencia, esto no fue un problema para la matemática.

En 1988, el matemático Peter Aczel publicó el libro *Non well founded sets*. Allí, planteó la teoría **ZFA** (Zermelo-Fraenkel con Anti-Fundación) como una extensión de la mencionada teoría **ZFC**. Al notar lo antinatural que resultaba modelar sistemas circulares en **ZFC**, se propuso quitar el Axioma de Fundación que complicaba las operaciones. Dentro de **ZFA** valen los axiomas de la teoría de conjuntos clásica exceptuando **FA**, que es reemplazado por el Axioma de Anti-Fundación (**AFA**, por sus siglas en inglés). La nueva teoría garantiza la existencia de los conjuntos no necesariamente bien fundados, los cuales son llamados *hypersets*, y crea un entorno consistente para modelar fenómenos circulares.

AFA

Aunque Aczel aplicó **ZFA** para modelar procesos computacionales, Barwise y Lawrence Moss observaron el potencial de la teoría para modelar la circularidad en computación, filosofía y en paradojas. En sus palabras, "(...) La teoría de conjuntos no bien fundados nos da una herramienta hermosa, coherente y poderosa con la cual estudiar fenómenos circulares en todos los dominios (...)" (Barwise & Moss, 1996, p. 64). El enfoque práctico de Barwise y Moss busca poner a disposición herramientas y esquemas de pensamiento para modelar circularidad dentro de teoría de *hypersets*. En el libro *Vicious Circles* formalizaron varias aplicaciones de la teoría: en las Ciencias de la Computación con *streams*[5] y máquinas de estado

[5]Un *stream* sobre un conjunto A se define como un par ordenado $s = (a, s')$, donde $a \in A$ y s' es otro *stream*. Para modelar *streams* circulares y mantener la estructura de pares ordenados, inevitablemente se requiere de la teoría de conjuntos no bien fundados (Barwise & Moss, 1996,

finito, y en Filosofía con la teoría de verdad y referencia.

En este respecto, Barwise y Etchemendy estudiaron exhaustivamente la paradoja del Mentiroso. Enfocaron su investigación desde la teoría de verdad situacional, la cual concibe que la veracidad de una proposición no es solo una propiedad de esta, sino una relación entre la proposición y un contexto sobre el cual se hace la afirmación. Mediante el estudio de las bases estructurales del Mentiroso con modelos en la teoría de *hypersets*, encontraron que el razonamiento paradójico tradicional ignora un parámetro implícito: la situación sobre la que se hace la proposición (Barwise & Etchemendy, 1987, p. 173).

El trabajo de Barwise, junto a Moss y Etchemendy, salió de la teoría de conjuntos convencional y dejó de lado la corriente que negaba la entidad de la autorreferencia dentro de las bases matemáticas. Los autores consideran que, mientras que las paradojas fueron cruciales para la construcción de la teoría de conjuntos, la teoría de *hypersets* es una oportunidad para hacer explícitas las suposiciones que dan lugar a paradojas dentro de un entorno matemático diseñado para explorar las circularidad (Barwise & Etchemendy, 1987, p. 8). La relevancia de los *hypersets* para las paradojas semánticas, por lo tanto, no es la de resolverlas en el sentido de anular la circularidad, sino de modelarla de manera consistente para extraer de ella un diagnóstico lógico y semántico profundo.

2.2.2. Priest: paradojas autorreferenciales del *Inclosure Scheme*

El dialeteísmo[6] considera a la contradicción como un campo de estudio inexplorado. Partiendo de este marco, Priest estudió la estructura de estos últimos y fundamentó que son objeto de contradicciones verdaderas (Priest, 1995, p. 4). Priest logró construir una caracterización, formalizada en el *Inclosure Scheme*, de la estructura subyacente de las paradojas de la autorreferencia y los límites conceptuales del pensamiento. El *Inclosure Scheme* expone una situación en donde paradojas, como la de Russell o el Mentiroso, generan contradicciones en sus límites.

Antes de presentar el *Inclosure Scheme*, es importante resaltar algunos puntos entorno a la investigación del infinito de la obra de Cantor. Los límites del pensamiento están intrínsecamente relacionados con el transfinito[7], pensando a cada ordinal como una barrera que se rompe al crecer en cantidad. Para cada conjunto,

p. 34).

[6]El dialeteísmo va más allá de la noción clásica de verdad y considera que hay falsedades que son verdaderas. Las lógicas no clásicas y paraconsistentes le brindan al dialeteísmo un espacio formal (Priest, 1995).

[7]Un número transfinito puede pensarse como un número cardinal o un número ordinal y es usado en la comparación de conjuntos infinitos. En el caso de los ordinales, se usa la cadena ordenada de $\omega, \omega+1, \ldots, \omega+\omega, \ldots$ (Davis, 2018).

hay un conjunto más grande. En general, esto significa que existen infinitos más grandes que otros. La técnica de diagonalización de Cantor puede ser pensada como la construcción de un elemento que rompe ciertos límites, dando lugar a un absurdo.

Diagonalización

En esencia, dada una lista de cierto tipo tenemos una construcción que define un nuevo objeto del mismo tipo, a través de la destrucción sistemática de la posibilidad de la identificación de este con cualquier elemento de la lista. El nuevo objeto, hecho de cada elemento que no está en la diagonal, "diagonaliza" la tabla de asignaciones (Priest, 1995, p. 132). Para bajar a tierra este método de prueba poderoso pero complicado de entender al principio, veamos el siguiente ejemplo. Supongamos que queremos ver si existe una relación uno a uno entre cualquier conjunto x y $\mathscr{P}(x)$. Primero, tomamos cada elemento del conjunto $x = \{\clubsuit, \diamondsuit, \heartsuit\}$ y cualquier conjunto de $\mathscr{P}(x)$ y construimos la siguiente asignación:

$$\clubsuit \leftrightarrow P_1 = \{\heartsuit, \diamondsuit\},\ \diamondsuit \leftrightarrow P_2 = \{\heartsuit, \diamondsuit, \clubsuit\},\ \heartsuit \leftrightarrow P_3 = \{\heartsuit, \clubsuit\}$$

Luego, construimos la siguiente tabla marcando con $+$ o $-$ si cada conjunto contiene o no su propia etiqueta. Los símbolos en la diagonal están encerrados en círculos para darles énfasis.

	$\{\heartsuit, \diamondsuit\}$	$\{\heartsuit, \diamondsuit, \clubsuit\}$	$\{\heartsuit, \clubsuit\}$
$\clubsuit$	$\ominus$	$+$	$-$
$\diamondsuit$	$+$	$\oplus$	$-$
$\heartsuit$	$-$	$-$	$\oplus$

Podemos dar un elemento nuevo δ que no se encuentra en el isomorfismo planteado entre paquetes y etiquetas. Este elemento es negación de todos los valores que representa la diagonal: si $\clubsuit$ no está en P_1, entonces está en δ; si $\diamondsuit$ está en P_2, entonces no está en δ y así con $\heartsuit$.

	δ
$\clubsuit$	$+$
$\diamondsuit$	$-$
$\heartsuit$	$-$

Notar que el nuevo conjunto $\delta = \{\clubsuit\}$ no está presente en la asignación inicial. Entonces, hay un conjunto no listado, por lo cual $\mathscr{P}(x) > x$ y no existe un isomorfismo entre x y $\mathscr{P}(X)$. Este resultado puede generalizarse a cualquier conjunto. Al aplicar la técnica sobre cualquier conjunto finito o infinito, siempre se termina obteniendo un conjunto por fuera de la asignación (Davis, 2018, pp. 57-59).

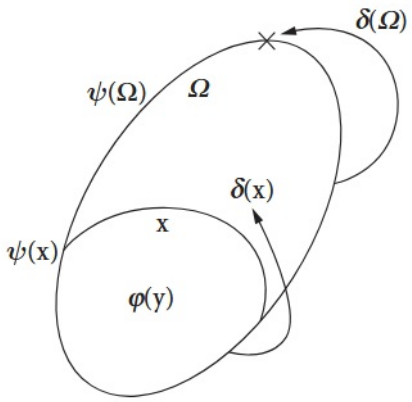


Figura 3: Diagrama del *Inclosure Scheme*

Russell, además de encontrar su famosa paradoja mediante la diagonalización, estableció una relación con la familia de paradojas autorreferenciales y la diagonalización. Partiendo de este postulado, Priest formalizó la estructura de las paradojas de autorrefencia de la siguiente manera:

Inclosure Scheme

$$
\begin{array}{lll}
(1) & \Omega = \{y; \varphi(y)\} \text{ existe y } \psi(\Omega) & \text{Existencia} \\
(2) & \text{si } x \subseteq \Omega \text{ y } \psi(x) & \\
 & \quad (a)\ \delta(x) \notin x & \text{Transcendencia} \\
 & \quad (b)\ \delta(x) \in \Omega & \text{Clausura}
\end{array}
\tag{1}
$$

Ya que al aplicar (a) y (b) sobre Ω tenemos que $\delta(\Omega) \notin \Omega \wedge \delta(\Omega) \in \Omega$, podemos decir que el diagonalizador se encarga de echar a $\delta(x)$ del conjunto Ω y a la vez encerrarlo dentro de Ω. Notar que φ es una propiedad de todos los elementos de la totalidad Ω y ψ una propiedad que algunos elementos de la totalidad cumplen.

Priest considera que la contradicciones al límite del pensamiento tienen una estructura bipartita común: una totalidad de objetos (Ω) y una operación (δ) que genera un objeto que está tanto dentro como fuera de esa totalidad (Priest, 1995, p. 4). En la Tabla 1 podemos ver solo algunas de las paradojas que encajan en el *Scheme* y solo unas ciertas formas de aplicarlo. Todas las paradojas de autorreferencia tradicionales se ajustan al *Inclosure Scheme* tienen la misma estructura subyacente y, por lo tanto, pertenecen a la misma familia. Estas paradojas dependen de un diagonalizador aplicado a una totalidad en su límite.

En lo que respecta a la paradoja de Russell, tenemos que la propiedad $\phi(y)$ asegura que los elementos pertenecen al conjunto de todos los conjuntos V. A su

[8] ! es una abreviatura de $\delta(\Omega) \in \Omega \wedge \delta(\Omega) \notin \Omega$.

Paradoja	$\delta(x)$	$\psi(x)$	$\phi(y)$	Ω	$\delta(\Omega) \in \Omega!$[8]
Russell	$\rho_x(\bigstar)$	$\lambda x.x$	$y \in V$	V	$\rho_V \in V!$
Mentiroso	$\alpha = \langle \alpha \notin x \rangle$	x es definible	y es verdadero	Tr	$\alpha(Tr) \in Tr!$
Heterological	$\langle v \in x \rangle$	x es definible	$\neg y$ sat y	Het	$\langle v \in Het \rangle \in Het!$

Tabla 1: *Inclosure Scheme* aplicado a las paradojas del Mentiroso, Heterological y Russell

vez, $(\bigstar)\rho_x = \{y \in x : y \notin y\}$ es el conjunto russelliano sobre $x \subseteq \Omega$. El diagonalizador nos indica que $\rho_V = \{y \in V : y \notin y\}$ pertenece a V y a la vez no. Priest da una idea de la demostración para ver que ρ_x cumple las propiedades de Trascendencia y Clausura (Priest, 1995, pp. 141-142). Así, podemos ver que la paradoja de Russell se acopla al *Inclosure Scheme*.

Sobre la paradoja semántica del Mentiroso, Tr es el conjunto de todas las oraciones verdaderas y, al aplicar el diagonalizador α sobre este conjunto, obtenemos una oración que afirma su falsedad. Luego, el *Scheme* construye la contradicción de que tal oración pertenece (Clausura) y no pertenece (Trascendencia) a Tr.

En el caso de la paradoja de Heterological, Het es el conjunto de las palabras heterológicas y, al tomar una palabra en ese conjunto mediante el diagonalizador, el *Scheme* construye una oración heterológica que afirma que es heterológica. Dado que los elementos de este conjunto cumplen la propiedad $\phi(y)$ que indica que no se aplican a sí mismos, esta oración lleva a una contradicción (Trascendencia y Clausura). No se entrará en detalle de cómo se llega a las propiedades de Trascendencia y Clausura en cada caso, pero tener en mente que se aplica el T-Schema de Tarski[9].

En la Figura 3 vemos que en los bordes de la contradicción se abre la posibilidad de un orden extraño del mundo, donde los objetos se retraen sobre sí mismos[10]. La persistencia de la contradicción implica que la dinámica de las paradojas autorreferenciales está en constante tensión y retorno. De algún modo, los ciclos extraños de Hofstadter no son ni más ni menos que la representación de este salto hacia afuera de la totalidad que vuelve a su origen.

[9]El T-Schema de Tarski es la siguiente definición de verdad dentro de un sistema formal: $T(\ulcorner\varphi\urcorner) \leftrightarrow \varphi$, donde T es el predicado que representa la verdad y $\ulcorner\varphi\urcorner$ es un código o nombre para una oración cualquiera φ (Bolander et al., 2006, pp. 4-5).

[10]En la Ontología Orientada a Objetos de Timothy Morton, los objetos son *inclosures*. En sus palabras, "(...) Las cosas también contienen otras cosas que no son estrictamente ellas mismas – del mismo modo en que una cebra no se reduce a sus átomos, desde cierto punto de vista, y aún así una cebra está compuesta solamente por esos átomos en particular. (...) [Los objetos] son "cerrados" – una cebra no es una jirafa – y, al mismo tiempo, no están cerrados – contienen cosas que no son ellas mismas. (...) Ser una cosa (...) es estar atravesada por la contradicción. (...)" (Morton, 2013, p. 28).

2.2.3. Grim y Rescher: diagramas estructurales de la oscilación paradójica

La investigación de Grim y Rescher enfatiza el rol de la reflexividad como hilo conductor entre estructuras conceptuales de un amplio territorio filosófico. En *Reflexivity: From Paradox to Consciousness*, se proponen alcanzar un entendimiento lo suficientemente profundo de las estructuras reflexivas como para graficar sus componentes (Grim & Rescher, 2012). Entre las instancias de patrones de reflexividad que buscan descomponer y reconstruir, se encuentran las paradojas autorreferenciales. De acuerdo con los autores, las paradojas generan inconsistencias problemáticas que van más allá de las contradicciones en sí mismas; de hecho, la oscilación infinita entre soluciones contradictorias es la magia que las hace interesantes (Grim & Rescher, 2012, p. 11).

En esta investigación, se encontraron y graficaron tres ingredientes fundamentales de las paradojas: la reflexividad, que permite construir un elemento autorreferencial; la negación, ubicada en una parte crucial de la proposición desencadenante; y una condición *todo y solo*, limitante del alcance de dicha proposición. La combinación de estas tres condiciones es lo que encapsula el mecanismo abstracto y simple que subyace a la oscilación conceptual de fenómenos reflexivos semánticos o de teoría de conjuntos.

Gracias a esta nueva caracterización de los problemas conceptuales de las paradojas y sus consecuencias inevitables, es posible trazar aspectos comunes entre ellas y construir un esquema general[11]. El siguiente principio sintetizador de la estructura de las paradojas es aplicable a cualquier relación R y cualquier tipo de entidad:

Principio 1 (Primer Principio de Consistencia Autorreferencial). No hay nada que esté en la relación R con todos y solo aquellos objetos que no están en relación R consigo mismos[12] (Grim & Rescher, 2012, p. 18).

El diagrama de la Figura 4 muestra el absurdo que surge de suponer que existe un elemento X que es la excepción al principio. Allí se observa una oscilación infinita entre dos proposiciones, mucho más directa de digerir que el principio anterior. A la derecha, "el conjunto X está en relación R consigo mismo". Esta se representa con una flecha que indica la pertenencia del conjunto a sí mismo. A la izquierda, "el conjunto X no está en relación R consigo mismo", representada del mismo modo, pero con la flecha tachada para simbolizar la negación de la relación principal. Dos flechas discontinuas encierran a los dos conjuntos en un eterno retorno, simbolizando las inferencias lógicas que llevan a la oscilación. En

[11]Notar que lo mismo podría decirse del *Inclosure Scheme*. Los autores mencionan que sus diagramas están fuertemente inspirados en la Figura 3.

[12]De manera más precisa, no existe ningún elemento x tal que, para todo y tal que $(y,y) \notin R$, se de que xRy solo con elementos del tipo y. Notar que R es una relación no reflexiva y no simétrica (Grim & Rescher, 2012, p. 17).

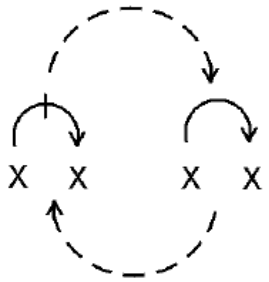


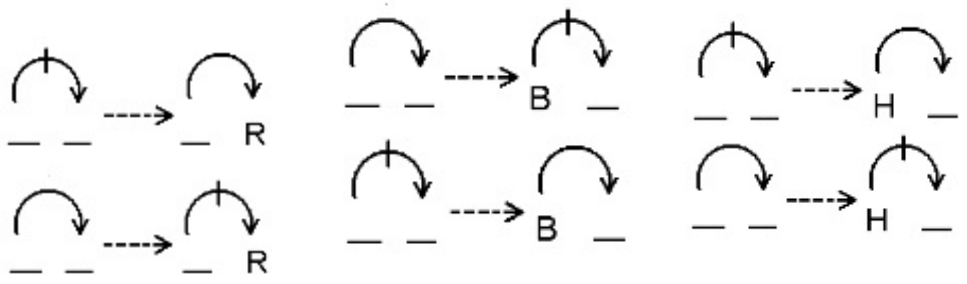
Figura 4: Diagrama estructural de las paradojas

general, la flecha $\curvearrowright$ representa la relación principal de la proposición; la flecha tachada $\not\rightarrow$ indica su negación; y la flecha discontinua $\dashrightarrow$, la inferencia lógica.

En las Figuras 5a, 5b y 5c tenemos elementos tentadores para hacer uso del esquema general. Una aplicación tentadora de este esquema es la paradoja de Russell. En la Figura 5a se observa el gráfico del conjunto russelliano. Similarmente al lenguaje de programación Haskell, las rayas horizontales indican que este espacio puede ser llenado por cualquier elemento, siempre y cuando sea el mismo. La flecha representa pertenencia de conjuntos. El primer diagrama, representa la proposición totalizante: "todo conjunto que no pertenece a sí mismo cumple que pertenece al conjunto R". En el segundo diagrama, se representa la proposición excluyente: "solo los conjuntos que no pertenecen a sí mismos pertenecen al conjunto R". Luego de retratar el conjunto R, podemos usarlo como un relleno de los espacios en blanco de las condiciones que lo definen. Al hacerlo, obtenemos que R pertenece a sí mismo si y solo si R no pertenece a sí mismo. Es decir, una oscilación parecida a la Figura 4.

Grim y Rescher opinan que la separación entre categorías de paradojas – como semánticas y de conjuntos – y un tipado estricto de los diagramas harían que el enfoque sea tan específico como para perder de la posibilidad de estudiar la reflexividad a modo de fenómeno general en diversos contextos (Grim & Rescher, 2012, p. 17). De este modo, se enfatiza el potencial generalizador de la estructura propuesta para las paradojas. La Figura 5b caracteriza al barbero B que afeita a todos y solo a aquellos que no se afeiten a sí mismos. Al rellenar los blancos con la misma persona B obtenemos una oscilación precisamente como la estructura general de la Figura 4. Análogamente, se genera un ciclo como este al rellenar los espacios del diagrama en la Figura 5c por el conjunto H.

Una caracterización en términos de proposiciones, centrándose en su sujeto y el predicado que se le atribuye a este, demostraría la misma estructura oscilatoria que la Figura 4. Los autores toman el principio generalizador de este tipo de oraciones como el siguiente:



(a) Conjunto R de conjuntos que no se contienen a sí mismos

(b) Conjunto B representativo del barbero que solo afeita a quienes no se afeitan a sí mismos

(c) Conjunto H compuesto por las palabras que no se aplican a sí mismas (heterologicas)

Figura 5: Definiciones de estructuras reflexivas a través de diagramas. Son aplicables al esquema general de la dinámica oscilatoria de paradojas de teoría de conjuntos

Principio 2 (Segundo Principio de Consistencia Autorreferencial). No hay predicado que esté en relación R con todas y solo aquellas oraciones cuyo predicado no está en relación R con su sujeto[13] (Grim & Rescher, 2012, p. 17).

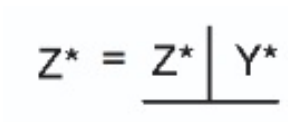
Para diagramar esto, se usa un símbolo divisor $\perp$. A la izquierda, va el sujeto y a la derecha va el predicado de la oración que se busca graficar.

Suponiendo que existe el predicado que el Principio 2 niega, podemos diagramar la condición totalizante y la condición excluyente que lo componen. Dado un elemento Y^* de tipo Y que es la excepción al principio. En la Figura 6b se diagraman estas propiedades. Rellenando los espacios en blanco con la oración Z^* de la Figura 6a – que se tiene a sí misma como sujeto y a Y^* como predicado — obtenemos la oscilación de la Figura 6c.

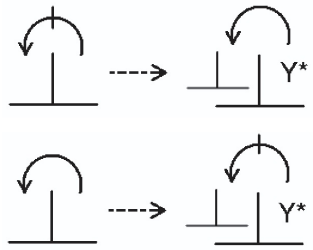
Salvo por eso las condiciones que generan la paradoja son similares a las paradojas más simples: la condición todo y solo se construye con el predicado Y^*. Como vimos, este predicado solo aplica a *todas y solo* a aquellas oraciones cuyo predicado no aplica al sujeto. Así, ambos diagramas son manifestaciones de un patrón reflexivo general que es responsable de las dinámicas de oscilación independientemente de su instanciación en conjuntos u oraciones (Grim & Rescher, 2012, p. 29).

En lo que respecta a la paradoja del Mentiroso, la relación principal que posibilita su estructura oscilatoria está en la aplicación del predicado sobre el sujeto. Para representar esta relación, usamos la oración de la Figura 7 que se tiene como

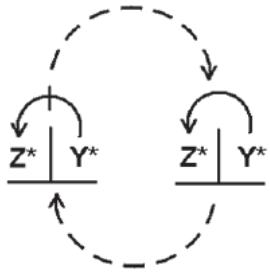
[13]Podemos expresar esto más formalmente. Sea R la relación entre sujeto y predicado, tal que xRy significa que el sujeto x está en relación R con el predicado y. Para toda oración z y su predicado y que no cumplen zRy, no existe ningún predicado p tal que cumpla zRp.



(a) Oración Z^* de tipo Z que se tiene a sí misma como sujeto



(b) Predicado Y^* de tipo Y que se aplica a todos y solo a aquellas oraciones donde su predicado no se aplica a su sujeto



(c) Estructura de las paradojas

Figura 6: Diagramas de la dinámica oscilatoria de las paradojas mediante oraciones con sujeto y predicado

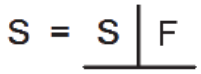


Figura 7: Oración S que se tiene a sí misma como sujeto y que se le aplica el predicado "es falsa", representado por F

sujeto y, a través de su predicado "es falso", se niega. Tomando Z^* y Y^* como S y F, respectivamente, podemos construir un diagrama como el de la Figura 6c. Subyacen a este las condiciones totalizantes y excluyentes presentes en la Figura 6b y generadas simplemente instanciando el diagrama general.

También, aunque no utiliza la misma estructura que los diagramas generales, podemos graficar la paradoja del Mentiroso Dual. Esta usa las siguientes oraciones:

- S1: "La oración S2 es falsa"
- S2: "La oración S1 es verdadera"

Sin elaborar demasiado, para cada frase se elaboran condiciones totalizantes y excluyentes en relación a la falsedad de las oraciones. Estas se pueden observar en la Figura 8 con el diagrama de tal paradoja.

Retomando lo explicado al comienzo de la sección, la investigación de Grim y Rescher no solo tiene como objetivo comprender la reflexividad en las paradojas sino en otras áreas del espectro filosófico. En efecto, los diagramas permiten proyectar el alcance de las paradojas sobre teoremas limitantes del siglo XX. En ocasiones se han establecido similitudes estructurales entre el argumento básico

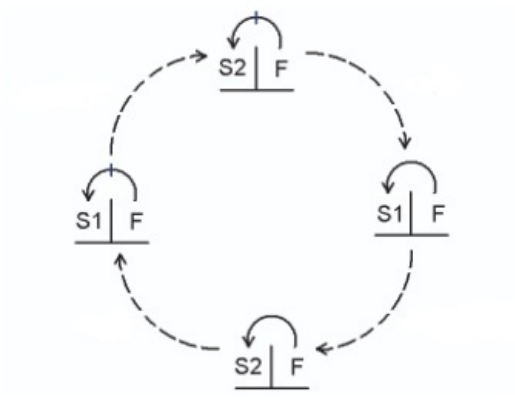
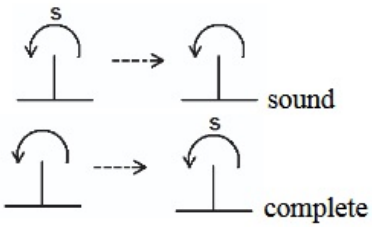
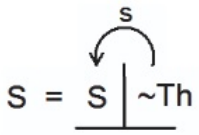


Figura 8: Diagrama de la paradoja del Mentiroso Dual

de la prueba del Teorema de Incompletitud de Gödel y la dinámica de la paradoja del Mentiroso. En estas analogías, la demostración de la incompletitud de los sistemas formales se deriva del absurdo que produce la fórmula autorreferencial "Esta fórmula no es un teorema". No entraremos en detalles de la dinámica de la prueba pero sí en su vínculo con las técnicas previas.



(a) Oraciones que expresan las nociones de corrección y completitud en un sistema formal *s*



(b) Oración *S* que se tiene a sí misma como sujeto y cuyo predicado de no ser un teorema es derivable en el sistema formal *s*

Figura 9: Diagramas de condiciones y proposiciones fundamentales para la dinámica oscilatoria de la prueba del Teorema de Incompletitud de Gödel.

Según esta forma de estructurar la prueba de Gödel, ningún sistema formal del tipo considerado puede ser, a la vez, correcto y completo. Un sistema es correcto si todo lo que se puede demostrar como teorema dentro del sistema es verdadero – es decir, si todos los teoremas son verdaderos bajo su interpretación. Un sistema es completo si toda proposición expresable en el sistema que sea verdadera bajo su interpretación aparece efectivamente como un teorema (Grim & Rescher, 2012,

p. 41). Si todo lo demostrable es verdadero, entonces siempre habrá más verdades de las que se pueden derivar dentro del sistema. O sea, cualquier sistema formal es incompleto.

En este respecto, vale destacar que las condiciones de corrección y completitud la Figura 9a son las que generan la oscilación infinita y, por lo tanto, el resultado de que un sistema no puede ser completo y correcto a la vez. Se usa la misma flecha de antes y una nueva que marca la relación de derivación dentro de un sistema formal s. En la Figura 9b hay una forma general para la oración inicial "Esta fórmula no es un teorema". Solo refiere a la noción de teorema, mediante la negación Th, y derivación en un sistema formal cualquiera[14].

También, los autores modelan con sus diagramas la prueba por el absurdo del *Halting Problem*. Evocan este teorema crucial para la computabilidad del siglo XX de la siguiente manera: no existe una Máquina de Turing que diga si cualquier Máquina de Turing se detendrá partiendo de cierto *input* o no (Grim & Rescher, 2012, p. 48). Los componentes del diagrama incluyen Máquinas de Turing y su código. En $\perp$, la parte izquierda es para el *input* y la derecha es para la máquina de Turing. La relación principal en este caso es detención, representada con una flecha con h y, en ocasión, hx donde x es el número de *output*. En este caso, la reflexividad es causada cuando una máquina toma su propio código como *input*. Sumado a esto, la condición todo y solo propicia la oscilación.

El punto central a destacar de la obra de Grim y Rescher es la convicción de que representar la naturaleza viva y cíclica de la reflexividad, mediante diagramas con ciertos ingredientes indispensables, posee un gran potencial para ayudar a comprender y establecer patrones en su estructura. Los diagramas en sí mismos son una estrategia simple de explicitar los componentes esenciales de las paradojas de autorreferencia y pruebas que juegan con la verdad de proposiciones con estructuras similares. Identifican en ellos patrones comunes y proponen una visión que los proyecta a la conciencia, el libre albedrío y las oraciones autorreferenciales tanto en el ámbito epistemológico como en el lingüístico, en torno a la referencia y la semántica. Además, modela el *Cogito* cartesiano y desarrolla aspectos del pensamiento nihilista.

[14]También, Grim y Rescher realizan un diagrama más similar a la demostración original, introduciendo aspectos técnicos con nuevos símbolos en los diagramas (Grim & Rescher, 2012, pp. 177-183). Esta forma sitúa a la fórmula S dentro de un sistema numérico e incorpora la numeración de Gödel. En esencia, S se auto-aplica el predicado $\sim Th$, forzando al sistema a enfrentarse a una verdad matemática que no puede demostrar.

2.3. El modelado computacional de problemas filosóficos

2.3.1. La programación como exploración

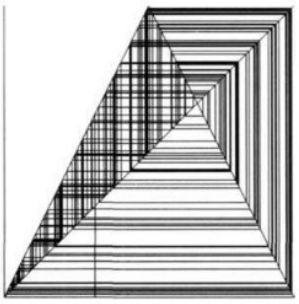
La construcción de modelos computacionales de sistemas lógicos puede ser vista como una forma de trabajo experimental, en el sentido amplio de la palabra. A partir de una formalización inicial, es posible construir un algoritmo que simule el comportamiento del sistema en cuestión, permitiendo realizar experimentos computacionales. Dado que se definen los límites y objetivos del código sobre la marcha, es un proceso intermitente de prueba y error hasta ajustar las intenciones iniciales a los resultados de las simulaciones. Las conclusiones se sacan a partir de resultados abiertos, generados por las simulaciones en constante evolución y modelos computacionales tentativos.

Desde una perspectiva epistemológica, los modelos computacionales cumplen funciones explicativas, exploratorias y predictivas (Primiero, 2020, p. 239). Estas simulaciones no solo ilustran las propiedades y patrones del modelo, sino que también pueden sugerir nuevas hipótesis. Incluso cuando fallan, su fracaso es informativo; al hacer evidentes las limitaciones, permiten revisar supuestos y afinar teorías. La investigación valora los resultados de la exploración en su totalidad, ya sean experimentos inesperadamente fructíferos o que encuentran dificultades inesperadas. El *trade off* de los modelos computacionales, vistos como una simplificación de los conceptos teóricos para simular distintos escenarios, es maleabilidad vs. exclusión: cuanto más flexible y maleable es un modelo, más riesgo hay de dejar afuera dimensiones relevantes del fenómeno (Grim et al., 1998, p. 11). Por ello, es crucial definir el nivel óptimo de abstracción.

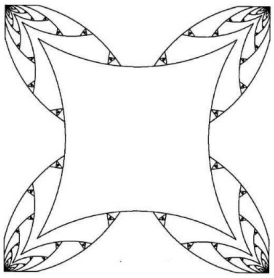
La filosofía de la ciencia, entre otros problemas de interés, investiga qué son y qué información construyen los modelos computacionales. Desde una mirada interdisciplinar, la filosofía podría nutrirse de las simulaciones computacionales, no solo por su significado en relación al conocimiento científico, sino por su capacidad de elucidar problemas filosóficos. Este enfoque tiene la capacidad de forzar la precisión y claridad. Tal cambio de metodología puede revelar rincones inexplorados y reformular preguntas viejas (Grim et al., 1998, p. 10). Es esto lo que plantea Grim, junto con Gary R. Mar y Paul St. Denis, en *The Philosophical Computer*.

Catorce años antes de la publicación de *Reflexivity: From Paradox To Consciousness*, Grim et. al realizaron modelos computacionales de las paradojas autorreferenciales en términos de *fuzzy logic*[15], fractales y otras representaciones

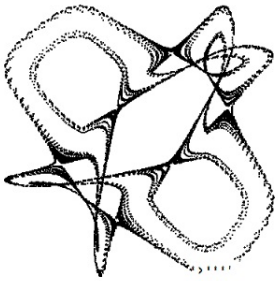
[15]Las lógicas de muchos valores son lógicas no clásicas. Difieren de la lógica clásica por el hecho fundamental de que no restringen el número de valores de verdad a solo dos: permiten un conjunto más grande W de grados de verdad (Gottwald, 2022). En este caso, el conjunto W es el conjunto infinito $[0,1]$. 0 representa "totalmente falso", 1 representa "totalmente verdadero", y los otros valores se refieren a grados de verdad parcial.



(a) Técnica gráfica *cobwebbing* o telaraña para valor inicial $r = 0{,}34$ en la fórmula iterativa del Mentiroso Caótico



(b) Fractal de tiempo de escape de la fórmula iterativa del Mentiroso Dual Caótico



(c) Atractor extraño de la fórmula iterativa del Mentiroso Caótico Triple

Figura 10: Algunos resultados de las simulaciones computacionales realizadas por Grim, Mar y Denis.

visuales de la teoría del caos[16]. También, investigaron las aplicaciones epistemológicas de estos modelos en teoría de juegos. A modo de ejemplo, realizaron simulaciones de estrategias para el dilema del prisionero, lo cual permitió estudiar la cooperación social en un modelo de sociedad Hobbesiana.

Los autores consideran que los experimentos filosóficos no son nada nuevo; de hecho, han construido conocimiento y sido foco de discusión de la filosofía. Tomemos de ejemplos la alegoría de la caverna de Platón, el experimento de la habitación china de Searle y los cerebros dentro de una cubeta de Putnam. Recuperando su opinión, "(...) Así como los experimentos mentales son más que dispositivos expositivos, también pueden serlo los modelos (...)" (Grim et al., 1998, p. 9).

En este caso, los experimentos no dependen exclusivamente de las intuiciones filosóficas sino que la computadora permite simular el resultado dentro de una concepción simplificada de la problemática original. Al incorporar la computadora a la investigación filosófica, el trabajo consiste en adaptarla a un ambiente nuevo, abstracto y algorítmico. En este contexto, la preocupación de Grim et. al es explorar los patrones infinitamente intricados del caos escondidos dentro de las paradojas (Grim et al., 1998, p. 15). En otras palabras, no buscan dar una solución

[16]La teoría del caos es el estudio de los sistemas dinámicos con comportamiento no determinista, sensible a condiciones iniciales y caóticos. Un fractal es una figura geométrica que exhibe una autoafinidad infinitamente compleja a medida que aumentan los niveles de expansión (Grim et al., 1998, p. 15). El *feedback* continuo, el efecto mariposa y otras características esenciales al caos aparecen en este tipo de figuras recursivas.

ni un patrón simple y predecible sino más bien usar la teoría del caos para estudiar la autorreferencialidad en sí misma (Grim et al., 1998, p. 31).

Dentro del esquema de *fuzzy logic*, resulta natural la creación de oraciones que se refieran a sus valores estimados. En este marco, la paradoja del Mentiroso Caótico se define como la proposición que afirma: "esta declaración es tan verdadera como se estima que es falsa" (Grim et al., 1998, p. 32). Así, comienzan expandiendo los operadores lógicos básicos a un contexto plurivalente y generalizando la teoría de verdad de Tarski para permitir grados de verdad[17]. A esto le llaman *semántica dinámica*.

Dentro del panorama de la semántica dinámica, los autores pudieron modelar la autorreferencia del Mentiroso Caótico usando funciones iterativas. Cada iteración n genera la estimación del valor de verdad de una proposición, representado por x_n. El algoritmo para calcular los valores sucesivos x_{n+1} se da mediante la siguiente función de iteración:

$$x_{n+1} = 1 - |(1 - x_n) - x_n| \qquad (\dagger)$$

Dado que el objeto de la teoría del caos es el comportamiento de secuencias iterativas como estas, podemos establecer que la semántica de Tarski en el contexto de *fuzzy logic* proporciona el marco matemático necesario para que los fenómenos caóticos puedan manifestarse dentro de los sistemas autorreferenciales. Esta fórmula es, de hecho, una función caótica conocida como *tent map*. El gráfico de la Figura 10a fue construido con la técnica de la teleraña correspondiente a tal fórmula †. El caos generado por esta función se caracteriza por tener una dependencia sensible de las condiciones iniciales, por lo que la estimación inicial genera resultados no determinísticos. Además, al consistir en experimentos de muchas iteraciones que se retroalimentan, este tipo de simulaciones generan imágenes caóticas y fractales.

Es posible crear un fractal usando la computadora, mediante los llamados diagramas de tiempo de escape. Por ejemplo, el conjunto de Mandelbrot[18]. El algoritmo en cuestión se basa en contar el número de iteraciones que necesita un punto inicial para que su trayectoria se mueva más allá de un umbral previamente definido. A grandes rasgos, el color asignado a cada punto refleja el número de iteraciones necesarias para que el sistema alcance la estabilidad o se escape más allá de un umbral (Grim et al., 1998, p. 43).

La idea general de las funciones iterativas – y una característica de los sistemas caóticos – es el *feedback* entre las estimaciones obtenidas en los pasos consecutivos. En el caso de la paradoja del Dualista Caótico (Grim et al., 1998, p. 39), las

[17]Traducen el condicional de Tarski al cálculo proposicional multivaluado usando el bicondicional de Jan Lukasiewicz (Grim et al., 1998, p. 20).

[18]https://mandelbrot.site/

dos oraciones mutuamente referenciales son:

X : X es tan verdadera como Y.
Y : Y es tan verdadera como X es falsa.

Dentro de la *semántica dinámica*, las fórmulas que representan esta oscilación son

$$\begin{cases} x_{n+1} = 1 - |y_n - x_n|, \\ y_{n+1} = 1 - |(1 - x_n) - y_n|. \end{cases} \qquad (\ddagger)$$

El fractal de tiempo de escape presente en la Figura 10b –generado con las ecuaciones semánticas ‡, la elección de un umbral y condiciones iniciales – es un vínculo visual entre fractales y el Dualista Caótico. De este modo, el diagrama de tiempo de escape es un retrato estático de la dinámica interna de la fórmula de la paradoja bajo iteración, revelando estructuras fractales que hacen visible la complejidad de la autorreferencia. También, las funciones iterativas de este tipo pueden utilizarse para graficar la dinámica no-linear de los sistemas caóticos en forma de atractores extraños[19]. En la Figura 10c podemos ver un objeto caótico tridimensional. Es creado a partir de tres funciones iterativas, cuyo valor es un cuarto de la diferencia entre el valor estimado de las otras (Grim et al., 1998, p. 51).

La computadora como espacio exploratorio permitió la creación de experimentos filosóficos para mostrar los patrones no lineales de las dinámicas autorreferenciales en lógicas plurivalentes. Los autores defienden a la computadora como una herramienta esencial para desenmascarar este vínculo y su gran potencial como laboratorio filosófico. El hecho de que las imágenes fractales y los atractores extraños de la Figura 10 surjan de manera tan natural de un análisis semántico de valores infinitos de las paradojas – dicho en palabras de Grim et. al – "(...) no solo parece ofrecer una extraña belleza, sino también la promesa intrigante de verdades más profundas (...)" (Grim et al., 1998, p. 52).

2.3.2. El horizonte interdisciplinario y pedagógico del modelado filosófico

En la sección anterior se expuso que la programación como exploración que no solo inspira nuestro trabajo en el ámbito del estudio de la autorreferencia, sino que también abre otras preguntas en torno a las aplicaciones de la propuesta computacional y filosófica: ¿Qué otro tipo de colaboraciones interdisciplinares se puede generar con el uso de modelos computacionales de sistemas lógicos? ¿A qué áreas

[19]Un atractor es la región del espacio hacia donde se dirigen los estados futuros del sistema dinámico. La parte de extraño surge porque estas órbitas están encerradas en esta región de manera caótica y generan fractales (Grim et al., 1998, p. 40).

se puede expandir el laboratorio filosófico de Grim et. al? ¿Cuál es la utilidad pedagógica de estos modelos en la enseñanza de la filosofía y la lógica? Lejos de contestar estas preguntas, buscamos que resuenen en el lector y desaten la curiosidad acerca del potencial de este tipo de proyectos.

Una de las posibles ramas de expansión del laboratorio filosófico es la dimensión pedagógica. De hecho, el trabajo expuesto previamente de Grim et. al surgió en el Laboratorio Lógico, un espacio pedagógico en la universidad para explorar lógica a través de la computación (Grim et al., 1998, p. 11). Entre los autores que fueron nombrados, Barwise y Etchemendy han realizado varias contribuciones al uso de computadoras para mejorar el aprendizaje de la lógica[20]: *Tarski's World*, para la enseñanza del lado semántico de la lógica de primer orden; *Hyperproof*, una introducción al pensamiento analítico y *Turing's World*, una presentación de las Máquinas de Turing.

Una propuesta reciente es la de Daniel Lim en *Philosophy Through Computer Science*, que gira en torno a usar programas de Python para aclarar o elaborar problemas filosóficos. Con la esperanza de crear un puente entre las humanidades y las disciplinas STEM, Lim desarrolló proyectos de programación que abordan problemas filosóficos clásicos. Algunas de las actividades consisten en vincular la manipulación gráfica de imágenes con el escepticismo sobre el mundo externo y el Juego de la Vida con el libre albedrío. El objetivo general es brindar a los estudiantes de carreras de grado la oportunidad para desarrollar las habilidades de pensamiento crítico, comunicación precisa y resolución de problemas (Lim, 2023, p. 2).

2.4. Próximos pasos en el laboratorio filosófico

Avanzaremos manteniendo en cuenta los estudios de las paradojas de autorreferencia de las secciones anteriores. De Priest, Grim y Rescher, aprendimos que graficar la dinámica de las paradojas pueden enseñarnos mucho sobre ellas y que honrar su estructura nos brinda la posibilidad de pensarlas como el resultado de una receta, un algoritmo o una función. Gracias a Barwise, Moss y Etchemendy, encontramos que el universo de los *hypersets* permite generar grafos de conjuntos no bien fundados y que este ambiente fue fructífero para el estudio de paradojas autorreferenciales. Sumando a los modelos de *The Philosophical Computer*, no podemos dejar de lado a la computadora como herramienta para la investigación de la autorreferencia. Tal como señalan los autores, su intención es que estos experimentos funcionen como punto de partida para nuevas indagaciones filosóficas: "(...) Nuestra esperanza es que estos ensayos exploratorios también estimulen a otros a emprender exploraciones sobre distintas preguntas filosóficas muy distin-

[20]https://openproof.gradegrinder.net/

tas (...)" (Grim et al., 1998, p. 10).

Expandiremos este espacio lúdico sin restricciones creativas para probar y analizar modelos computacionales de problemas filosóficos. Con ese espíritu en mente, surge una pregunta inevitable: ¿Qué ocurriría si utilizáramos modelos computacionales de *hypersets* para visualizar la dinámica de las paradojas y de los componentes que posibilitan su existencia? ¿Podríamos identificar patrones transversales entre las distintas teorías y los diagramas generados?

3. Librería Hypersets

En esta sección se presenta el contenido de la librería Hypersets, implementada en Haskell. Las explicaciones de sus funcionalidades buscan ser una introducción amigable a la teoría de *hypersets*. Esta herramienta computacional es una nueva adquisición del laboratorio filosófico, que nos permite modelar sistemas reflexivos y explorar sus propiedades. El link al repositorio de Github está en el Anexo A. No ahondaremos en demostraciones de **AFA** ni en casos borde de la teoría, debido a que no es necesario para nuestros modelos ni experimentos.

La organización de la explicación seguirá el esquema de la Figura 11. Mientras que en la sección 3.1 se desarrolla la relación entre *hypersets* y grafos dirigidos etiquetados, en la sección 3.2 se presenta la noción de conjunto solución de un sistemas de ecuaciones dentro del universo de conjuntos no bien fundados. Finalmente, se explora el *pipeline* sistema $\rightarrow$ conjunto $\rightarrow$ grafo en la sección 3.3.

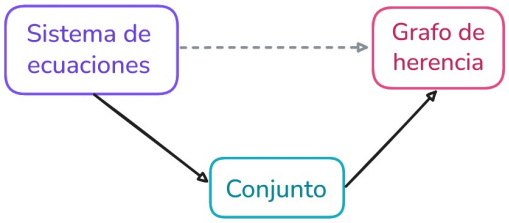


Figura 11: Esquema teórico de la transformación de sistema de ecuaciones a grafo en la teoría de *hypersets*.

3.1. De conjunto a grafo

Como mencionamos en la sección 2.2, esta teoría de conjuntos admite conjuntos no bien fundados. La teoría de *hypersets*, al ser una extensión de $\mathbf{ZFC}^{-}$ (Teoría de conjuntos **ZFC** sin los axiomas de Fundación o Anti-Fundación), nos permite utilizar las operaciones de teoría de conjuntos sin realizar cambios (Barwise &

Etchemendy, 1987, p. 42). Para modelar los conjuntos de la teoría de Aczel, utilizaremos el tipo de datos `data HFS t = U t | S [HFS t]` (Tarau, 2008, p. 2). Dentro de esta representación, un conjunto se escribe "desenrrollando" la relación de herencia entre sus elementos. Estos últimos pueden ser *urelementos* (`U`) o conjuntos (`S`)[21]. En la siguiente tabla podemos ver algunos ejemplos de conjuntos bien fundados.

Conjuntos	`HFS String`
$\emptyset$	`S [ ]`
$x = \{w, z\}$	`x = S [U "w", U "z"]`
$x = \{w, z\}, w = \{v\}$	`x = S [S [U "v"], U "z"]`

Particularmente, la tercera fila tiene una forma parecida a un sistema de ecuaciones algebraico. Al definir un conjunto estamos definiendo una ecuación dentro del universo de los *hypersets*. Ya hablaremos de esto más adelante.

Algunas opciones para modelar un conjunto no bien fundado como $x = \{x, z\}$ son `x = S [S [U "x"], U "z"]` y `x = S[U "x", U "z"]`. Sin embargo, es una circularidad de juguete y, quizás, podamos construir una simulación más cercana a lo cíclico en Haskell. Algo a tener en cuenta es que buscamos *simular* la circularidad en lugar de emplearla directamente y sin escrúpulos[22], como sería es caso de `q = S [q]`. En cierto modo, esta limitación de nuestros modelos los hace más manejables.

Para lograr representar que un conjunto se refiere a sí mismo de una manera flexible y conceptualmente fiel, podemos expandir la estructura `HFS` de la siguiente manera:

Listing 1: Tipo de datos de conjuntos hereditarios finitos con `ID`

```
type ID = Int
data RefHFS t = RefS ID [RefHFS t] | RefU (t, ID)
```

La clave es que cada elemento tiene un número identidad, cuya asignación comienza en 0 y parte desde afuera hacia adentro del conjunto. Esta propuesta provee un marco accesible para definir referencias y auto-referencias.

Dentro de la teoría de *hypersets*, los grafos etiquetados tienen un rol fundamental. Un grafo es simplemente una tupla $\langle G, E \rangle$ donde G es un conjunto de

[21] Notar que trabajamos sobre el universo $\mathscr{U}$ de los *urelementos*, entendidos como entidades atómicas que no son conjuntos y que pueden aparecer como elementos de conjuntos. Por eso, usamos "átomo" y "*urelemento*" de manera intercambiada.

[22] Una forma de modelar la paradoja de Russell genera un bug en Haskell: https://downloads.haskell.org/ghc/latest/docs/users_guide/bugs.html#bugs-in-ghc.

Ecuaciones	`RefHFS String`
$x = \{x\}$	`RefS 0 [RefU ("X",0)]`
$x = \{x, z, w\}$	`RefS 0 [RefU ("X",0), RefU ("Z",1), RefU ("W",2)]`

nodos (de cualquier tipo) y $E \subseteq G \times G$ una relación binaria. Si $(a,b) \in E$, decimos que hay una arista de a a b (Barwise & Moss, 1996, p. 119). A partir de aquí, la relación E será representada con $\rightarrow$. Un grafo etiquetado es lo mismo salvo que con una función que asigna conjuntos a cada nodo (Barwise & Moss, 1996, p. 125).

Definición 3.1 (Grafo etiquetado)**.** Sea $A \subseteq \mathscr{U}$. Un *(labeled graph)* $\overrightarrow{G} = \langle G, \rightarrow, l \rangle$ sobre A es una 3-upla tal que $\langle G, \rightarrow \rangle$ es un grafo, y $\ell : G \rightarrow \mathscr{P}(A)$.

En este respecto, un grafo bien fundado es uno que no tiene caminos infinitos (Aczel, 1988, p. 4). Contrariamente, un grafo no bien fundado es uno con ciclos. Pueden ser directos sobre un nodo o formados por un camino de nodos.

En nuestra implementación, usaremos el tipo `Graph`, que asocia a cada vértice una lista de los nodos con los cuales se conecta de modo directo («Data.Graph», 2025). A diferencia de la teoría, el conjunto de vértices se compone por números naturales en lugar de conjuntos. También, incorporamos grafos etiquetados con `LabGraph`, compuestos por `Graph` y con el sinónimo de tipo `Labeling`, que es la función de etiquetado que asigna a cada vértice un conjunto de etiquetas (HaskellWiki, 2025). Notar que estas etiquetas van a poder ser conjuntos `S` o *urlementos* `U`, a diferencia de la teoría donde las etiquetas de un nodo son un conjunto dentro de $\mathscr{P}(A)$.

Listing 2: `Graph` y `LabGraph`

```haskell
type Vertex = Int
type Graph = Array Vertex [Vertex]
type Edge = (Vertex, Vertex)
type Bounds = (Vertex, Vertex)

type Labeling a = Vertex -> HFS a
data LabGraph n = LabGraph Graph (Labeling n)
```

Es posible usar los nodos y las aristas del grafo para visualizar conjuntos y la pertenencia de elementos (Barwise & Etchemendy, 1987, p. 39). Cada conjunto puede ser fácilmente visualizado con un grafo construido en base a la relación de pertenencia invertida. Es parecido a un diagrama de Hasse – por ejemplo, $\mathscr{P}(\{a,b\})$ – solo que en lugar de representar relaciones de orden se representan relaciones de herencia.

Una forma de construir un grafo canónicamente es comenzando con el conjunto deseado a y considerar todos los conjuntos hereditarios como nodos. Las aristas del grafo conectan a los conjuntos con sus miembros hereditarios (Barwise & Etchemendy, 1987, p. 37). La idea es que, en un grafo canónico, los hijos de un nodo a son exactamente los elementos de a (Barwise & Moss, 1996, p. 123).

Definición 3.2 (Grafo canónico etiquetado)**.** Sea c cualquier conjunto y sea $G_c = \langle G, \rightarrow, \ell \rangle$ el grafo cuyo conjunto de nodos es $\mathrm{TC}(\{c\})$. Esto es, el menor conjunto transitivo que incluye a c. Recordemos que un conjunto c es transitivo si cada elemento de c tiene la propiedad de que sus elementos también pertenecen a c (Barwise & Moss, 1996, p. 14). Por lo tanto, G es un conjunto de conjuntos.

La relación $\rightarrow$ se define por

$$a \rightarrow b \quad \text{si y sólo si} \quad b \text{ es un conjunto y } b \in a,$$

y la función de etiquetas está dada por $\ell(a) = a \cap \mathscr{U}$. En otras palabras, se etiqueta con un conjunto de los *urlementos* que pertenecen a a.

Observar que la relación de aristas es la *conversa* de la relación de pertenencia en G_c (Barwise & Moss, 1996, p. 128).

Entonces, un grafo canónico concentra la relación de pertenencia de un conjunto. Los grafos no bien fundados son generados por un conjunto no bien fundado (Aczel, 1988, p. 6). Lo mismo sucede con los bien fundados. Por ello, el grafo anterior es no bien fundado.

Ahora bien, para obtener el conjunto que es el esqueleto de un grafo canónico etiquetado, definimos la decoración de grafos etiquetados (Barwise & Moss, 1996, p. 123).

Definición 3.3 (Decoración etiquetada)**.** Sea $A \subseteq \mathscr{U}$. Una *decoración* de un grafo etiquetado $\overrightarrow{G}$ sobre A es una función $d : G \rightarrow \mathscr{V}[A]$[23] tal que para todo $g \in G$

$$d(g) \equiv \{d(h) \mid g \rightarrow h\} \cup l(g)$$

La *decoración* de un nodo se define como el conjunto de las decoraciones de sus hijos (Barwise & Moss, 1996, p. 4).

Notar que si G_a es un grafo canónicamente etiquetado, entonces su decoración d es la identidad sobre el conjunto de nodos G. En efecto, como cada nodo b tiene por etiqueta al propio conjunto b, el cálculo del decorado se reduce a la definición del conjunto. De este modo, $d(b) = b$.

[23] Donde $V[A]$ es el universo de *ZFA* con todos los conjuntos conformados con átomos incluídos en A (Barwise & Etchemendy, 1987, p. 49). Como dato extra, **AFA** garantiza que cada conjunto en $V[A]$ puede obtenerse como parte de la solución de un único sistema de ecuaciones que utiliza elementos de A como átomos. O sea, $V[A] = V_{afa}[A]$ (Barwise & Moss, 1996, p. 75).

Ejemplo 3.1 (Adelanto de grafo generado por la librería Hypersets para $a = \{c, \{c\}\}$)**.** Dado el siguiente conjunto (donde c actúa como átomo)

$$a = \{c, \{c\}\}$$

Otra forma de escribirlo es descomponerlo en sus elementos hereditarios.

$$\begin{cases} a = \{c, b\} \\ b = \{c\} \end{cases}$$

Al asignar a cada variable un *hyperset*, tenemos un sistema de ecuaciones dentro del universo de *hypersets*. Dado este sistema de ecuaciones y el *labeling*, nuestra librería va a representar a mediante la construcción de un grafo de herencia con átomos incluidos. Una característica fundamental de nuestra librería es que incluye a los *urelementos* como nodos y como etiquetas.

Aunque ya sabemos que los nodos se van a representar con enteros, facilita la comprensión de nuestra implementación si imaginamos que el conjunto de nodos está compuesto por $TC(\{a\}) \cup A$. Supongamos que las aristas representan la relación de pertenencia directa. Si el etiquetado consiste en la función identidad, obtenemos el grafo de la Figura 12. Cada nodo está etiquetado con a, con un átomo o con el elemento correspondiente de TC({a}).

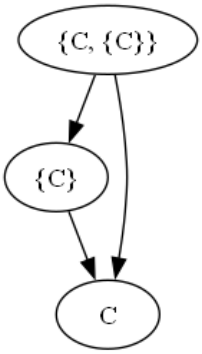


Figura 12: Grafo de herencia de a

Como podemos ver, la raíz del grafo está etiquetada con el conjunto a y las bases de la jerarquía de herencia están etiquetadas con los átomos de a. Notar que este grafo es bien fundado (Barwise & Moss, 1996, p. 126).

A diferencia de los grafos etiquetados de la definición, nuestra implementación permite que las etiquetas sean elementos de cualquier tipo de elementos (ya sean conjuntos o *urlementos*). La definición de decoración que implementamos usa la estructura de unión comentada, admitiendo la unión de un conjunto con un

urlemento. Nuestro algoritmo se diferencia de la teoría en que, al llegar a un nodo sin hijos, se lo decora con su etiqueta.

Siguiendo esta idea, nuestra librería va a generar el decorado de este grafo como lo que encontramos a continuación:

$$\begin{aligned} d(\{c,\{c\}\}) &= \{d(\{c\}), d(c), l(\{c,\{c\}\})\} = \{c,\{c\}\} \\ d(\{c\}) &= \{d(c), l(\{c\})\} = \{c\} \\ d(c) &= l(c) = c \end{aligned}$$

Acá podemos ver las maneras complementarias de representar la relación de pertenencia invertida que posibilitan los grafos de herencia y los decorados. Notar que cada nodo ha sido etiquetado con su correspondiente decorado. O sea, se cumple que $d(b) = l(b) = b$ para cada nodo b. Recordemos nuevamente que los nodos en nuestra implementación serán números.

Ejemplo 3.2 (Adelanto del grafo generado por la librería para $\Omega = \{\Omega\}$)**.** Sea Ω el siguiente conjunto no bien fundado

$$\Omega = \{\Omega\}$$

Dada esta ecuación, podemos construir un siguiente grafo de herencia donde el conjunto Ω es el único nodo y el *labeling* es la función identidad

A partir de ahora nos referiremos al grafo de herencia asociado a Ω como G_Ω.

Para formalizar aún más la relación entre conjuntos y grafos, introducimos el concepto de *accessible pointed graph*.

Definición 3.4 (Accessible pointed graph)**.** Un *accessible pointed graph* (abreviado como *apg*) es un grafo etiquetado $\overrightarrow{G} = \langle G, \rightarrow, \ell, \top_G \rangle$ con un nodo raíz $\top_G$ tal que existe un camino finito hacia cualquier nodo partiendo de este (Barwise & Moss, 1996, p. 129).

Definición 3.5 (Visualización)**.** Dado un conjunto x, una visualización de x es cualquier *apg* cuya decoración del nodo raíz sea x (Devlin, 1993, p. 154). En otras palabras, decimos que un *apg* $\overrightarrow{G}$ es la visualización del conjunto $d(\top_G)$ (Barwise & Moss, 1996, p. 129).

Hay varias formas de construir una visualización. Algo a destacar es que un grafo canónico G_x es una visualización de $d(x)$ o, lo que es lo mismo, del conjunto x (Barwise & Moss, 1996, p. 129). De hecho, los grafos canónicos son las visualizaciones elegidas por los autores (Omodeo E. G., 2017, p. 74).

Nuestras visualizaciones, siguiendo las decisiones de diseño tomadas y los modelos desarrollados, son grafos de herencia como los que vimos en los Ejemplos 3.1 y 3.2. Precisamente, el grafo del Ejemplo 3.1 es nuestro modelo de la visualización del conjunto a. De entre las visualizaciones posibles, nos gustan estas porque cada nodo está etiquetado con el elemento correspondiente a la clausura transitiva del conjunto que buscamos visualizar. Así, son más fáciles de comprender[24].

Para construir nuestro decorado, usamos la función `computeDecorations`. Asocia cada vértice con su decorado, representado con un conjunto `HFS` simple. Para ello, aplica la función `decorate` sobre cada vértice y aplica una traducción de la definición matemática de decorado. La función resultante sirve para construir el *labeling* de una visualización de ese conjunto.

Listing 3: `decorate` con operaciones de conjunto `HFS`

```
decorate :: Graph -> Labeling String -> [Vertex] -> Vertex -> HFS
    String
decorate gr label visited v
      | v `elem` visited = U ("d(" ++ show(label v) ++ ")")
      | null children = label v
      | otherwise = unionHFS (label v) (S childDecs)
      where
            children = gr ! v
            visited' = v : visited
            childDecs = map (decorate gr label visited') children
```

Con respecto al manejo de ciclos, funciona para el caso de grafos no bien fundados ya que identifica y corta los ciclos al guardar los nodos ya visitados. Notar que el decorado se construye mediante la operación `unionHFS`. Esencialmente, realiza concatenación de listas de conjuntos `HFS` sin repetir elementos. Para mayor legilibilidad, conviene usar la función `computeDecorationsShort`. Esta versión se construye el decorado de manera casi idéntica a `computeDecorations` solo que `childDecs` es una lista de `U ("d(" ++ show x ++ ")")` para cada hijo de v.

[24]Terminan siendo grafos parecidos a los que arman Barwise y Etchemendy. Las etiquetas de los nodos son sus decorados y, a su vez, el conjunto de todos los decorados es la clausura transitiva del conjunto que se busca visualizar (Barwise & Etchemendy, 1987, p. 44). Así que nuestra librería tiene un enfoque híbrido, tomando aspectos de *Vicious Circles* y *The Liar*.

Para desarrollar este pasaje entre conjuntos y grafos, construimos las funciones `setToLabGraph` y `showLabGraphViz`. `setToLabGraph` se encarga de construir un grafo de herencia etiquetado a partir de un conjunto `RefHFS` y una función *labeling*. Internamente, se llama a `setToGraph`. Allí, `getChildren` es aplicada a cada elemento, siendo la pieza clave para construir el arreglo de vértices y sus hijos. Por último, `showLabGraphViz` escribe un archivo dot a partir de un grafo etiquetado, permitiendo la generación de una imagen.

Ejemplo 3.3 (Simulación del ejemplo 3.1)**.** Definimos el conjunto y elegimos un *labeling* de entre todos los posibles. Luego, realizamos la transformación a grafo etiquetado, según la herencia de miembros.

```
wellFoundedSet :: RefHFS String
wellFoundedSet = RefS 0 [RefU ("C",2),RefS 1 [RefU ("C",2)]]

wellFoundedLabeling :: Labeling String
wellFoundedLabeling 0 = S [U "C", S [U "C"]]
wellFoundedLabeling 1 = S [U "C"]
wellFoundedLabeling 2 = U "C"
wellFoundedLabeling _ = U "???"

let labgraph = setToLabGraph wellFoundedSet wellFoundedLabeling
```

Internamente, `setToLabGraph` usa `setToGraph`.

```
wellFoundedGraph = setToGraph wellFoundedSet
print wellFoundedGraph
```

Esto tiene como resultado

```
array (0,2) [(0,[2,1]),(1,[2]),(2,[])]
```

Podemos ver que lo que antes eran `IDs` ahora son los vértices del grafo. De esta manera, obtenemos un elemento de tipo `Graph` generado con un arreglo de hijos indexados por vértices. Con el grafo etiquetado `labgraph`, calculamos las decoraciones de los vértices.

```
let decorations = computeDecorations labgraph
mapM_ (\ (v, d) -> putStrLn \$ " " ++ show v ++ ": " ++ prettyHFS
    d) (assocs decorations)
```

Como podemos observar, los conjuntos obtenidos son idénticos a los de la resolución matemática (donde 0 es $\{c, \{c\}\}$, 1 es $\{c\}$ y 2 es c).

```
  0: {C, {C}}
```

```
1: {C}
2: C
```

Finalmente, construimos una visualización.

```
let pict = setToLabGraph wellFoundedSet wellFoundedLabeling
-- La siguiente linea es equivalente
-- let pict = setToLabGraph wellFoundedSet (decorations !)
let diagram = showLabGraphViz pict
writeFile "experiments/output\_chapter/well_founded_example.dot"
    diagram
```

Esta función genera un archivo dot asociado al mismo grafo del Ejemplo 3.1.

Ejemplo 3.4 (Simulación del ejemplo 3.2)**.** Definimos el conjunto Ω y un *labeling* como

```
omegaSet :: RefHFS String
omegaSet = RefS 0 [RefU ("X",0)]

omegaLabeling :: Labeling String
omegaLabeling 0 = U "X"
omegaLabeling _ = U "???"
```

Aplicando las funciones obtenemos este decorado `0: {X, d(X)}` y este grafo `array (0,0) [(0,[0])]`. El decorado del nodo es su etiqueta y el decorado de sus hijos, o sea, de sí mismo. Podría desenrrollarse esta circularidad como {X, {{X, {{X, {...}}}}}}. Podemos resumir eso en $d(X) = X$ (Barwise & Moss, 1996, p. 121). Además, se genera un archivo dot con la estructura de G_Ω.

La teoría de Aczel parte de la intuición de que un conjunto es una colección de objetos cuya relación de pertenencia puede ser representada por un único grafo (Barwise & Etchemendy, 1987, p. 37). En particular, dentro de **ZFC**$^-$ eso estaba claro: el Lema de Colapso de Mostowski dice que cada grafo etiquetado y bien fundado tiene un único decorado (Aczel, 1988, p. 4), de lo que se puede deducir que cada conjunto tiene una única visualización. El axioma **AFA** extiende este resultado a todos los grafos etiquetados (Aczel, 1988, p. 10):

Definición 3.6 (Formulación de **AFA** para grafos etiquetados)**.** Cada grafo etiquetado tiene una única decoración etiquetada.

Bajo **AFA** se tiene que cualquier grafo representa un único conjunto: su decorado. A partir de este resultado, es posible deducir que cada conjunto tiene una única visualización. Para que dos conjuntos sean distintos tiene que haber una

diferencia estructural importante que evite que sean representados por el mismo grafo etiquetado (Barwise & Etchemendy, 1987, p. 40). Aunque no entraremos en detalle, dos grafos son bisimilares si representan el mismo *hyperset* (Boscaratto et al., 2025, p. 4).

3.2. De ecuaciones a conjunto

Una de las preocupaciones de la teoría de *hypersets* es sentar las reglas sobre la solución de sistemas de ecuaciones dentro de este mundo. Al ser una extensión de la teoría de conjuntos común, debemos definir qué pasa con las soluciones de ecuaciones cíclicas como $q = \{x, q\}$. De hecho, una de las formas de presentar **AFA** es mediante el *Solution Lemma*. A grandes rasgos, garantiza que todo sistema de ecuaciones de conjuntos – incluso los que son autorreferenciales – tiene una única solución en la teoría de los *hypersets*. No son las ecuaciones de siempre como $x = 2y + 3$ sino que contienen conjuntos del lado derecho de la igualdad (Barwise & Moss, 1996, pp. 68-70). Bajo **FA**, los sistemas bien fundados siempre tienen una única solución; de hecho, esto es esencialmente una reformulación del propio Lema de Colapso de Mostowski (Barwise & Moss, 1996, p. 75).

Antes de definir el *Solution Lemma*, establecemos sus componentes clave (Barwise & Moss, 1996, p. 71).

Definición 3.7 (Sistema de ecuaciones)**.** Un sistema de ecuaciones es una tupla $\mathscr{E} = \langle X, A, e \rangle$ que consiste de un conjunto $X \subseteq U$, un conjunto A disjunto de X, y una función

$$e \colon X \to \mathscr{P}(X \cup A).$$

1. X se llama el conjunto de *indeterminadas* de $\mathscr{E}$ y A se llama el conjunto de *átomos* de $\mathscr{E}$.

2. Para cada $v \in X$, el conjunto

$$b_v := e_v \cap X$$

   se llama el conjunto de indeterminadas de las que v depende inmediatamente.

3. De manera similar, el conjunto

$$c_v := e_v \cap A$$

   se llama el conjunto de átomos de los que v depende inmediatamente.

4. Una *solución* de $\mathscr{E}$ es una función s con dominio X que satisface

$$s_x = \{s_y : y \in b_x\} \cup c_x,$$

para cada $x \in X$.

Una vez que tenemos **AFA** a nuestra disposición, definimos el conjunto de soluciones de cualquier sistema $\mathscr{E}$ de ecuaciones como simplemente el conjunto de todos los conjuntos de la forma s_x, donde s es la solución de $\mathscr{E}$ y v varía sobre las indeterminadas de $\mathscr{E}$. En esencia, esta definición plasma la intuición de que la solución de un conjunto x está compuesta por las soluciones de las incógnitas que dependen de x. Por ello, es similar al caso conocido de las ecuaciones algebraicas. Les invitamos a pensarlo de esa manera.

Ejemplo 3.5.

$$\begin{aligned} x &= \{x, y\} \\ y &= \{p, q, y, z\} \\ z &= \{p, x, y\} \end{aligned} \tag{2}$$

Esas son las instancias de e para cada elemento de $X \cup A$, donde $X = \{x, y, z\}$ y $A = \{p, q\}$. O sea, $e_x = \{x, y\}$, $e_y = \{p, q, y, z\}$ y $e_z = \{p, x, y\}$. Luego, las soluciones de cada variable indeterminada son $s_x = \{s_x, s_y\}$, $s_y = \{p, q, s_y, s_z\}$ y $s_z = \{p, s_x, s_y\}$ (Barwise & Moss, 1996, p. 70).

En Haskell, modelamos estas ecuaciones con el tipo de datos `System`, compuesto por `Equation` y `SetExpr`.

Listing 4: Tipo `System` compuesto por `Equation` y `SetExpr`

```haskell
type Variable = String
data SetExpr t = Ref Variable | Expr t | SetOf [SetExpr t]
  deriving (Show)
data Equation t = Equation Variable (SetExpr t)
     deriving (Show)
type System t = [Equation t]
```

Un sistema de ecuaciones es una lista de ecuaciones del tipo `Equation`, una para cada `Variable` del conjunto de las indeterminadas. Las ecuaciones usan el tipo `SetExpr`, que tiene `Ref`, `Expr` y `SetOf` como constructores. Usamos `Ref` para referirnos a otra variable, posibilitando ciclos y referencias. Para referirnos a un *urlemento* o átomo, usamos `Expr`. El constructor `SetOf` sirve para iniciar un conjunto.

Para construir la solución de un sistema, utilizamos la función `denoteSystem`. A partir de un sistema de ecuaciones y una variable de interés, se construye el

Sistema	System
$x = \{x\}$	`[ Equation "X"(SetOf [Ref "X"]) ]`
$\begin{cases} x = \{y,z\} \\ y = \{x\} \end{cases}$	`[ Equation "X"(SetOf [Ref "Y", Expr "Z"]), Equation "Y"(SetOf [Ref "X"]) ]`

conjunto solución de la variable de interés. La decisión de trabajar con una indeterminada especial viene de que se facilita el pasaje ecuaciones, solución y *apg*[25].

Internamente, `denoteSystem` utiliza la función `convertSetExpr`, comenzando por la variable raíz. El código es extenso por lo que no indagaremos en este. La lógica de la implementación es convertir los elementos del tipo `SetExpr` a su equivalente en `RefHFS`. Se crea un arreglo de tuplas que asocia cada `Variable` a un `ID`, iniciando en 0 desde la raíz del sistema. En líneas generales, esto nos permite convertir `SetOf` en `RefS`, `Ref Variable` en `RefS`, `Expr` en `RefU` y cortar los caminos de referencia infinitos cuando aparezcan, pues trabajamos con conjuntos hereditariamente *finitos* (Omodeo E. G., 2017, p. 74) (Aczel, 1988, p. 7) (Boscaratto et al., 2025, p. 3). Para manejar correctamente los ciclos, se utilizan listas auxiliares `visited` y `expanded` dentro del algoritmo de construcción de soluciones.

Ejemplo 3.6 (Sistema X = {X} y conjunto solución)**.** Podemos definir la ecuación para Ω como vemos a continuación:

```
omega :: System String
omega = [ Equation "X" (SetOf [Ref "X"]) ]

set = denoteSystem omega "X"
print set
```

```
RefS 0 [RefU ("X",0)]
```

Notar que se genera el conjunto utilizado en el Ejemplo 3.2.

Ejemplo 3.7 (Sistema A = {C, {C}} y solución)**.** Podemos definir un sistema de ecuaciones como el que vemos a continuación:

```
wellFoundedSystem :: System String
wellFoundedSystem =
  [ Equation "A" (SetOf [Expr "C", Ref "B"])
```

[25]Esta decisión se respalda en la teoría de *hypersets* ya que corresponde a un sistema de ecuaciones *pointed* (Barwise & Moss, 1996, p. 88). Estos sistemas tienen su versión en el mundo de los grafos cuando hablamos de un *apg* (Barwise & Moss, 1996, p. 129).

```
  , Equation "B" (SetOf [Expr "C"])
  ]

set = denoteSystem wellFoundedSystem "A"
print set
```

```
RefS 0 [RefS 1 [RefS 2 [RefU ("C",3)]],RefS 2 [RefU ("C",3)]]
```

Notar que se genera el conjunto utilizado en el Ejemplo 3.1.

Ahora bien, el *Solution Lemma* plantea lo siguiente (Barwise & Moss, 1996, p. 72):

Definición 3.8 (Solution Lemma)**.** Cada sistema de ecuaciones $\mathscr{E}$ tiene una única solución s.

Esto nos indica que en los Ejemplos 3.6 y 3.7 no solo contruimos las soluciones para cada sistema, sino las únicas posibles.

3.3. De ecuaciones a grafos

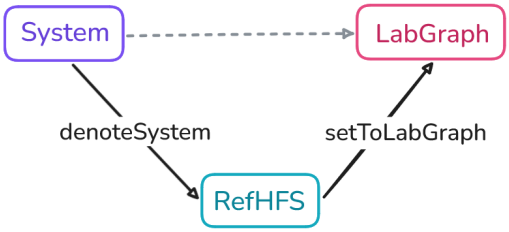


Figura 13: Esquema del modelo lógico de la teoría de *hypersets*.

Las transformaciones que vimos, de ecuaciones a conjuntos y de conjuntos a grafos, dan lugar al pasaje de ecuaciones a grafos de una manera bastante directa. Estas no son las únicas, sino que también son posibles los pasajes inversos, es decir, de grafo a conjunto y de conjunto a ecuaciones. El primero es debido a la ya mencionada única decoración de cada grafo. El segundo se da mediante la creación de un sistema de ecuaciones canónico[26], el cual además se puede pasar a un grafo canónico (Barwise & Moss, 1996, p. 128). Ya sentadas las bases de la teoría de *hypersets*, podemos pasar a acomodarnos con el panorama que nos

[26]Dado un conjunto a, un sistema canónico se construye con la clausura transitiva de a como las indeterminadas y la función la identidad como e.

plantea **AFA**: la traducción entre los tres mundos – conjuntos, grafos, ecuaciones – siempre es *posible y única*. De hecho, Barwise y Moss prueban la siguiente equivalencia (Barwise & Moss, 1996, p. 126):

Definición 3.9. Sea $A \subseteq U$. En $\mathbf{ZFC}^-$, son equivalentes las siguientes afirmaciones:

1. Todo sistema de ecuaciones

$$\mathscr{E} \equiv \langle X, A, e \rangle, \qquad \text{donde } e : A \to \mathscr{P}(X \cup A),$$

   tiene una única solución.

2. Todo grafo G sobre A tiene una única decoración.

Esto se debe que existe una correspondencia entre soluciones y decorado. Tanto la solución del sistema s como la decoración del grafo d son funciones que asignan un conjunto a cada nodo. Si una función s es una solución única para un sistema de ecuaciones E_G, entonces la función d definida como $d(a) = s(x_a)$ es una decoración única para el grafo G, y viceversa. Esto demuestra que la búsqueda de una solución al sistema es, matemáticamente, la misma tarea que la búsqueda de una decoración para el grafo.

Decidimos simular el pasaje de ecuaciones a conjuntos inspirados en el *Solution Lemma* y el pasaje de conjunto a grafo basados en la idea de que todo conjunto tiene una única visualización. Esto nos permite tener la comodidad de los grafos de relación de pertenencia invertida mientras aprovechamos la claridad presente en la formulación de sistemas de ecuaciones (Barwise & Moss, 1996, p. 130). Puede resultar redundante este pasaje, ya que vimos que sistemas de ecuaciones y decorados son inseparables, pero valoramos los distintos niveles de abstracción y detalle que proveen las dos formulaciones de **AFA**.

Ejemplo 3.8 (*Pipeline* completo de $X = \{\{0\}, \{0, X\}\}$)**.** A continuación, mostraremos un ejemplo nuevo que une los fragmentos y consecuencias de **AFA** que hemos propuesto. Partiremos de este sistema no bien fundado (Barwise & Moss, 1996, p. 91).

$$\begin{aligned} X &= \{Y, Z\} \\ Y &= \{0\} \\ Z &= \{0, X\} \end{aligned} \tag{3}$$

Notar que la solución de X es $s_X = \{\{0\}, \{0, X\}\}$. En teoría de conjuntos, tal conjunto es el par ordenado $\langle 0, X \rangle$.

Para minimizar la verbosidad del ejemplo, usamos `pipelineSystemToGraph`. Como lo dice su nombre, realiza el camino desde un sistema de ecuaciones, la solución y su grafo asociado.

```haskell
nonWellFoundedSystem :: System String
nonWellFoundedSystem =
  [ Equation "X" (SetOf [Ref "Y", Ref "Z"])
  , Equation "Y" (SetOf [Expr "0"])
  , Equation "Z" (SetOf [Expr "0", Ref "X"])
  ]

nonWellFoundedLabeling :: Labeling String
nonWellFoundedLabeling 0 = S [U "X"]
nonWellFoundedLabeling 1 = S [U "Y"]
nonWellFoundedLabeling 2 = S [U "Z"]
nonWellFoundedLabeling 3 = S [U "0"]
nonWellFoundedLabeling _ = S [U "???"]

pipelineSystemToGraph System "X" nonWellFoundedLabeling
```

Vamos a dar cuenta del *output* paso por paso. Primero, la solución que genera `denoteSystem` es idéntica en su forma a s_X.

```
== Sistema --> Conjunto ==

RefS 0 [RefS 1 [RefU ("0",3)],RefS 2 [RefU ("0",3),RefU ("X",0)]]
```

`setToLabGraph` genera una representación del conjunto, construido a partir de ese conjunto y el *labeling* ingresado. La función `showGrahpViz` hace la traducción a dot.

```
== Conjunto --> Grafo ==

array (0,3) [(0,[1,2]),(1,[3]),(2,[3,0]),(3,[])]

== Conjunto --> Grafo etiquetado ==

Grafo en el archivo experiments/output_chapter/non_well_founded.dot
```

Obtenemos el grafo de la Figura 14 :
El decorado de este grafo no es más que

$$
\begin{aligned}
d(0) &= \{d(1), d(2), x\} \\
d(1) &= \{d(3), y\} \\
d(2) &= \{d(3), d(0), z\} \\
d(3) &= \{0\}
\end{aligned}
$$

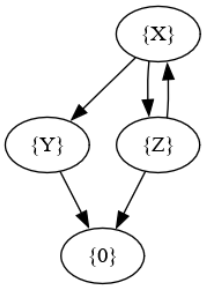


Figura 14: Grafo de herencia asociado a X etiquetado con `nonWell FoundedLabeling`.

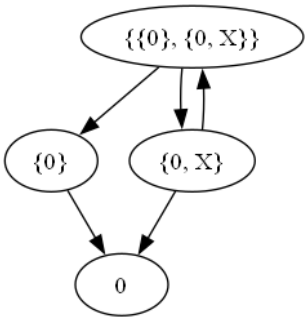
Lo cual coincide con el *output* de `computeDecorationsShort`.

```
= Decoraciones para el grafo =

  0: {X, d(1), d(2)}
  1: {Y, d(3)}
  2: {Z, d(3), d(0)}
  3: {0}
```

Es sútil pero estos nodos 0,1,2 y 3 representan, respectivamente, a las soluciones para las ecuaciones de X, Y, Z y al átomo 0. Como las aristas se construyen en base a la relación de pertenencia invertida entre estos nodos, el *pipeline* implementado en nuestra librería siempre genera un grafo de herencia a nivel estructural. Podemos etiquetar el grafo con total libertad, pero siempre tendrá la misma estructura hereditaria.

Si corremos `setToLabGraph` con el mismo conjunto pero etiquetando a cada nodo con el correspondiente elemento de la clausura transitiva de X o con su átomo 0, entonces construimos una visualización de X.



De este modo, la librería provee las funciones para generar la solución de sistemas y la representación de conjuntos como grafos.

3.4. Limitaciones y observaciones

Queremos destacar que simular la circularidad computacionalmente brinda una maleabilidad y facilidad importantes al proyecto. Esta maleabilidad conlleva falta de rango en el modelo. La limitación más evidente es la falta de ciclos, en lo estricto de la palabra; pues, si así los hubiera, no terminaríamos nunca de ejecutar nuestros experimentos. Sumado a esto, los tipos de datos propios `RefHFS` y `Graph` traen el costo de ineficiencia a la hora de recorrer elementos de la lista y arreglos que los conforman. Eso no afecta en ejemplos pequeños.

En pos de añadir un marco conceptual a la librería Hypersets, presentamos un breve análisis sobre las técnicas de diseño empleadas. En primer lugar, esta serie de módulos podría calificarse como un lenguaje de dominio específico embebido (EDSL, por sus siglas en inglés) dentro de Haskell, que es un lenguaje de propósito general (Gibbons & Wu, 2014, p. 1). Un posible refinamiento de un EDSL es un *deep embedding*. Consiste en la representación abstracta de la sintáxis como un tipo de datos, junto con algunas funciones que le asignan semántica a esa sintáxis a través de una transformación del tipo de datos (Gibbons & Wu, 2014, pp. 1-2). Al construir nuestros propios tipos de datos por cada representación de la teoría y las funciones `denoteSystem` y `setToPicture` que realizan el pasaje entre representaciones, no solo contruimos un EDSL sino que lo hicimos, en gran medida, con el enfoque de *deep embedding*.

Ahora bien, en un *shallow embedding* los términos en el EDSL son implementados directamente por su semántica, sin definir previamente una estructura de datos conceptual (Gibbons & Wu, 2014, p. 1). En nuestra implementación, incorporamos un *shallow embedding* mediante la decentralización de algunas operaciones com `computeDecorations` y el manejo de las *Labels* de manera holgada, entre otras herramientas como librerías importadas y funciones auxiliares.

Añadir la visión del enfoque *shallow* nos permite añadir nuevas representaciones, en cierto modo aisladas dentro del modelo, sin tener que adaptar los tipos de datos y las semánticas que los vinculan. Mientras que la noción *deep* centraliza las representaciones de datos de manera precisa, brindando expresividad al EDSL (Reynolds, 1978, p. 310). A pesar de implicar un proceso complicado y meticuloso, cada paso del proceso de diseño naturalmente llevó a utilizar los dos tipos de *embeddings*, ya que facilitan de distintas maneras el manejo sintáctico y el análisis teórico. No es sorpresa ya que en la literatura del área se concluye de diversas maneras que ambos enfoques están intrínsecamente ligados (Gibbons & Wu, 2014; Reynolds, 1978).

En lo que respecta a la eficiencia del código, una posible mejora es posibilitar la reutilización de cálculos en los casos de una operación recursiva, como la construcción de decorados. Además, sería importante considerar otros modos de almacenar información a la hora de recorrer listas de todo tipo (elementos de

un conjunto, listas de vértices, *labels*, decorados, nodos ya visitados, etc.). Otro punto para mejorar el uso de las funciones es hacer chequeos sobre los conjuntos o sistemas que se ingresan y hacer una interfaz gráfica dentro de Haskell, para lo cual podríamos basarnos en las ideas de Müjdat Pakkan y Varol Akman (Pakkan & Akman, 1995).

En relación con futuras ampliaciones de la librería, el trabajo de Paul Tarau implementa un modelo del *Ackermann encoding* en Haskell – o sea, representa la biyección entre conjuntos en **ZFC** y números naturales – y menciona su posible extensión a *hypersets*. Nuestra librería podría incorporar este tratamiento de los *hypersets*, haciendo uso de la investigación actual entorno al *Ackermann encoding* para conjuntos no bien fundados (Boscaratto et al., 2024). También, una ampliación relevante consiste de la implementación de bisimulación de grafos etiquetados (Dovier, 2015), lo que permitiría incorporar testing de corrección del modelo, por ejemplo verificando si dos grafos representan el mismo conjunto. En relación a esto, sería importante añadir funcionalidades para trabajar con grafos canónicos y sus decorados de manera ágil y cómoda.

4. Modelos y simulaciones

En este punto del trabajo estamos en condiciones de retomar la pregunta que nos trajo hasta acá: ¿Qué ocurriría si utilizáramos modelos computacionales de *hypersets* para simular la dinámica oscilatoria de las paradojas y de los componentes que posibilitan su existencia? Tomando como base la librería Hypersets y los antecedentes explicados en el capítulo 2, tenemos un amplio abanico de modelos por plantear y experimentos que ejecutar. En el Anexo A se encuentra el link al repositorio de Github con el código de los experimentos. A lo largo de la presentación de los modelos computacionales, justificaremos decisiones de diseño y plantearemos posibles aplicaciones.

Comenzaremos el capítulo creando modelos para proposiciones con y sin autorreferencia. Buscamos dar modelos que generalicen este tipo de estructuras. Para ello, nos basaremos en el modelo de 3-uplas planteado por Barwise y Etchemendy. Luego, serán empleados para modelar paradojas. Veremos que es posible establecer un paralelismo entre estos grafos y los diagramas de Grim y Rescher, como el de la Figura 6a. En particular, realizamos tres experimentos de este tipo.

Sumado a esto, en la sección 4.3 exploramos un modelo computacional para el *Inclosure Scheme*. Siguiendo la línea del diagrama de Priest, presente en la Figura 3, proponemos un modelo visualmente expresivo de la propiedad de Trascendencia y Clausura. Utilizamos la idea que acontece en todos los diagramas de Grim y Rescher, donde las flechas representan la aplicabilidad de un predicado. Esto implicó construir un prototipo de Hypersets con manejo de grafos con aristas

etiquetadas.

4.1. Proposiciones sin autorreferencia y sus paradojas

Tal como observamos en los diagramas de Grim y Rescher, una parte de los diagramas de sujeto y predicado consistía en las flechas tachadas o sin tachar, indicando el cumplimiento o no de este predicado por parte del sujeto. En relación a esto, Barwise y Etchemendy proponen otra forma de modelar proposiciones mediante la asignación de un lugar distinguido a cada componente, a saber: sujeto, predicado y valor de verdad del predicado (Barwise & Etchemendy, 1987, p. 42).

Supongamos que modelamos la proposición de que p tiene la propiedad E con la 3-upla $\langle E, p, 1\rangle$ y la proposición de que p no cumple E con la 3-upla $\langle E, p, 0\rangle$. Si el predicado E es "es verdadera", la 3-upla $q = \langle E, p, 0\rangle$ expresa la falsedad de p. Estas 3-uplas se consideran un par de pares en teoría de conjuntos.

$$
\begin{aligned}
\langle E, p, 0\rangle &\overset{(1)}{=} \langle E, \langle p, 0\rangle\rangle && \text{(por definición de 3-upla como tupla anidada)} \\
&\overset{(2)}{=} \{\{E\}, \{E, \langle p, 0\rangle\}\} && \text{(por definición de tupla: } \langle a, b\rangle = \{\{a\}, \{a, b\}\}\text{)} \\
&\overset{(3)}{=} \{\{E\}, \{E, \{\{p\}, \{p, 0\}\}\}\} && \text{(idem)}
\end{aligned}
$$

Acá, E y 0 son solo símbolos mientras que p es una proposición de las que modelamos. Notar que este enfoque implica el estudio de la referencia, una pieza clave para modelar proposiciones anidadas y circulares, más que de la expresión de predicados lógicos y su veracidad. En este sentido, encontramos una limitación: el modelo captura la estructura referencial de las proposiciones, pero no incorpora los valores de verdad, conectivos lógicos ni mecanismos de evaluación semántica necesarios para analizar contradicciones o derivar consecuencias propiamente lógicas.

Al permitirnos estudiar la estructura general de una proposición referencial mediante sistemas de ecuaciones, este modelo podría ser usado para construir sistemas de interreferencia más extensos que esta simple oración. Por tanto, tomamos la decisión de modularizar este código e incorporar un tipo de datos para este tipo de proposiciones.

```
data Sentence = Sentence
    {   name:: String, -- p
        subject :: String, -- q
        predicate :: String, -- E
        applicability :: String -- 0, 1
    }
```

Tomando este esquema referencial, es natural plantear las proposiciones mutuamente contradictorias del Mentiroso Dual. Usando el mismo predicado E = "es verdadera", tenemos las siguientes 3-uplas:

- $q = \langle E, p, 0 \rangle$
- $p = \langle E, q, 1 \rangle$

En nuestro modelo computacional, tenemos la posibilidad de construir este sistema de referencias cruzadas. El primer paso para ello consiste en la creación de un sistema de ecuaciones y un *labeling*, uno por cada proposición, y su uso como parámetro de entrada para el flujo `denoteSystem` $\rightarrow$ `setToLabGraph`. La función `sentenceToSystem` genera el sistema de ecuaciones asociado a las n-uplas. La clave de esta función es que deja una referencia colgada, abriendo paso a la proposición que llena ese lugar.

A la hora de crear un sistema de referencias complejo, solo basta con concatenar sistemas de ecuaciones asociados a distintas proposiciones. Algo parecido llevamos a cabo con los *labelings* de cada variable, al combinar los *labels* producidos por `sentenceLabeling` mediante `labelingCombined`.

```
liar1 :: Sentence
liar1 = Sentence "p" "q" "E1" "0"

liar2 :: Sentence
liar2 = Sentence "q" "p" "E2" "1"

dualLiarLabeling :: Labeling String
dualLiarLabeling = labelingCombined liar1 liar2

dualLiarSystem :: System String
dualLiarSystem = sentenceToSystem liar1 ++ sentenceToSystem liar2
```

En general, para que el sistema resultante refleje la idea de fusionar distintas estructuras referenciales, nos referimos de manera distinta a las ocurrencias del predicado E. Para el caso del Mentiroso Dual, el predicado E es representado por `E1` y `E2`. Una vez generados el sistema de ecuaciones y las etiquetas, ejecutamos el *pipeline* `denoteSystem` $\rightarrow$ `setToLabGraph`. El resultado obtenido se encuentra en la Figura 15.

Cabe destacar que el grafo que visualiza a una n-upla se vuelve ilegible rápidamente. Para facilitar la lectura de los grafos, las etiquetas fueron generadas de modo tal de estimular la reconstrucción mental de la estructura hereditaria de la n-upla. Los nodos sin hijos se etiquetan sin problema con correspondiente átomo. En el caso de los nodos con hijos, las etiquetas de cada nodo son abreviaciones

de los elementos correpondientes de la clausura transitiva de la n-upla que queremos visualizar. La información para acceder a las visualizaciones completas de los modelos de estas secciones se encuentra en el Anexo A.

Sobre esta base, podemos pensar en la construcción de la cadena de referencias característica de la paradoja de Yablo. Esta paradoja, recordemos, se compone por infinitas proposiciones, cada una haciendo referencia al resto de las subsiguientes y afirmando su falsedad. Un modelo de estas es el que tenemos a continuación:

$$s_i = \langle E,\, s_{i+1}, \ldots, s_n,\, 0 \rangle \qquad \forall i \in \mathbb{N} \text{ con } 0 < i < n$$

Para realizar un modelo computacional, construiremos estas proposiciones parcialmente incorporando la noción de profundidad para cortar la cadena en un punto deseado. Bajo nuestra concepción, la paradoja de Yablo con profundidad n está compuesta por las proposiciones s_1 hasta s_{n-1}. La función `sentenceYablo` arma la i-ésima proposición de la secuencia, que resulta como `s_i s_i+1 ... s_n Ei 0i`. El sistema de ecuaciones correspondiente contiene una referencia colgada a cada s_{i+1}.

Para lograr la cadena de referencias, llamamos a `yabloFamilySystem n` que concatena los sistemas desde s_1 hasta el límite s_{n-1}. Mediante `labelingCombinedList`, que combina una lista de `Labelings`, implementamos la función `yabloFamilyLabeling` para fusionar cada `yabloLabeling sentence` en un solo `Labeling` para un sistema con múltiples referencias. El siguiente experimento nos muestra la creación del sistema de ecuaciones para la paradoja de Yablo con profundidad 3.

```
sentYablo3 :: Sentence
sentYablo3 = sentenceYablo 1 3

sysYablo3 :: System String
sysYablo3 = yabloFamilySystem 3

labelingYablo3 :: Labeling String
labelingYablo3 = yabloFamilyLabeling 3
```

Corriendo el *pipeline* `denoteSystem` $\rightarrow$ `setToLabGraph`, obtenemos el resultado que observamos en la Figura 16. En definitiva, el modelo extiende una `Sentence` a una cadena de proposiciones donde cada una referencia a todas las siguientes, creando sistemas complejos para análisis de paradojas.

4.2. Proposiciones autorreferenciales y sus paradojas

Ahora bien, ya estudiamos las proposiciones que referencian a otras. Similarmente, para modelar las proposiciones autorreferenciales usaremos una 3-upla con

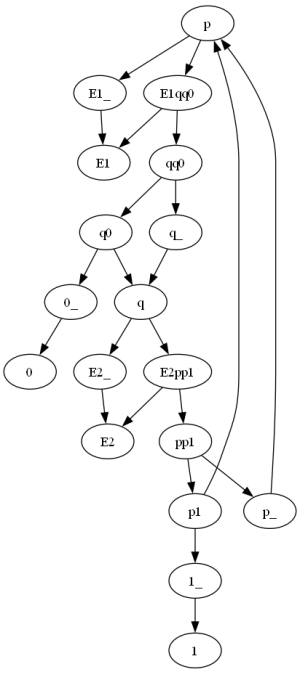


Figura 15: Grafo correspondiente al experimento del Mentiroso Dual

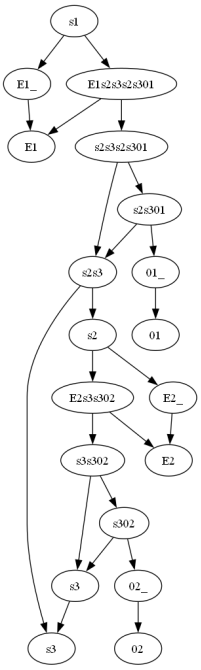


Figura 16: Grafo correspondiente al experimento de Yablo de profundidad 3

Figura 17: Modelos de paradojas sin autorreferencia

sujeto, predicado y valor de verdad de su aplicabilidad. La oración "Esta oración no cumple E" puede modelarse como $q = \langle E, q, 0 \rangle$. En otras palabras, la oración se niega a sí misma de manera iterativa: afirma no cumplir E, luego afirma que esa afirmación no cumple E, luego que la afirmación anterior no cumple E, y así sucesivamente.

Para expresar esta idea, podríamos concebir una instancia del tipo `Sentence`, donde `name` y `subject` son el mismo *string*. Puntualmente, `Sentence q q E 0`. Para evitar redundancias, inventamos un tipo `SelfRefSentence` con la siguiente estructura:

```
data SelfRefSentence = SelfRefSentence
    {   subject :: String, -- q
        predicate :: String, -- E
        applicability :: String -- 0, 1
    }
```

El sistema de ecuaciones que representa el par ordenado q es determinado por la función `selfRefToSystem`. A su vez, el `Labeling` de cada variable es elevado

por `selfRefLabeling`. Nuevamente, ingresamos el sistema de ecuaciones asociado a q al *pipeline* `denoteSystem` $\rightarrow$ `setToLabGraph`. Así pues, fabricamos el grafo de la Figura 18.

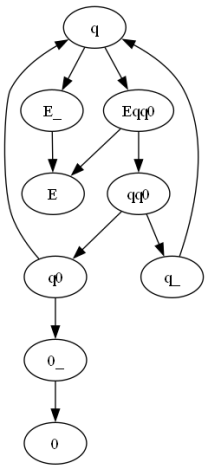


Figura 18: Grafo correspondiente al experimento del Mentiroso

Del mismo modo que la Figura 6a de Grim y Rescher posee la estructura general autorreferencial para las paradojas, el camino relatado para crear este grafo cíclico puede ser utilizado para esas mismas proposiciones. Tomemos, por ejemplo, la paradoja del Mentiroso. Si E = "es verdadera", entonces q representa "Esta oración es falsa". Ahí podemos ver definida la proposición $S = S \perp F$ que afirma su propia negación. En la segunda iteración, la oración vuelve a negarse. Podemos ver eso en ambos casos, ya sea en nuestros modelos y en el diagrama de Grim y Rescher: ""Esta oración es falsa" es falsa" y $S \perp F$ nos dice que S es falsa, respectivamente.

En lo que respecta a las paradojas de Russell, el Barbero y Heterological, es necesario hacer algunas aclaraciones. Sea E = "pertenece a sí mismo", aplicable a conjuntos de todo tipo. Primero, definimos el conjunto R como el conjunto donde cada elemento $x \in R$ cumple $x = \langle E, x, 0 \rangle$. Luego, la 3-upla $R = \langle E, R, 0 \rangle$ nos sirve para modelar la paradoja de Russell. Podemos aplicar la misma idea para la paradoja del Barbero y de Heterological.

Entonces, la 3-upla $q = \langle E, q, 0 \rangle$ nos sirve para representar la propiedad que cumplen los elementos del conjunto – no es verdadera, no pertenecen a sí mismos, no se afeitan a sí mismos y no pueden ser aplicados a sí mismos – y para representar la proposición paradójica inicial – S es falsa, R no pertenece a sí mismo, B no se afeita a sí mismo y H no pertenece a sí mismo.

Como observación, sería interesante modelar expresiones cuantificadas en pos de reflejar el significado de los predicados que se aplican a los sujetos a la vez que

los definen. Como una aproximación, los cuantificadores podrían modelarse como $\langle \forall, X, E, 1 \rangle$. Es más, una operación lógica como $\wedge$ podría ser modelada como $q \wedge p = \langle \wedge, q, p, 1 \rangle$.

4.3. Contradicción en el *Inclosure Scheme*

Como dijimos en la sección anterior, el modelo de tuplas alcanza su techo en la descripción de predicados. En el estudio de la estructura del *Inclosure Scheme*, aprovecharemos su complejidad para una alternativa más expresiva. El problema que abordamos en esta sección es cómo representar computacionalmente la aplicación de funciones dentro del *Inclosure Scheme* sin perder información estructural ni colapsar sus resultados en nodos opacos. El objetivo no es solo representar el *Inclosure Scheme*, sino disponer de un entorno computacional donde sus propiedades – incluidas las contradictorias – puedan explicitarse y analizarse.

En pos de enriquecer los modelos, proponemos las aplicaciones de funciones sobre conjuntos y *urelementos* (ver la definición en (1)). Las funciones booleanas no son solo ϕ y ψ, sino que también tenemos, de manera encubierta, $\in$ y $\subseteq$. Incidentalmente, podemos modelar la inclusión de conjuntos de la misma manera que antes. Sin embargo, en el caso de $\delta(x) \in \Omega$, donde un elemento de la imagen de δ pertenece a un conjunto, nos preguntamos si es suficiente modelarlo como un urlemento más o si es posible representar que es el resultado de la aplicación de δ sobre un conjunto.

En nuestro camino por ampliar la expresividad de los modelos computacionales que creamos, buscamos rellenar el hueco de propiedades de la aplicación de funciones con una 4-upla de la forma $c = \langle d, \Omega, \in, 1 \rangle$. Esta tupla y su visualización representan la propiedad de Clausura del diagonalizador δ. Lo mismo podría hacerse para la Trascendencia. Para la Existencia, definimos Ω como el conjunto que cumple ψ y se encuentra compuesto por elementos que cumplen ϕ. Con nuestro sistema de tuplas, eso sería $\langle \psi, \Omega, 1 \rangle$ y, para cada $x \in \Omega$, $\langle \phi, x, 1 \rangle$. Además, podemos expresar $x \subseteq \Omega$, dentro de un grafo de herencias, y la aplicación de ϕ sobre Ω como la tupla $\langle \phi, X, 1 \rangle$.

Una de las desventajas de este modelo es que cada concepto dentro del *Scheme* está separado. Se realiza una representación local de Existencia, Clausura, Trascendencia y sus componentes. Para obtener una representación global del *Scheme*, algo similar a la Figura 3, exploraremos una propuesta más flexible. Basándonos en la Figura 3, consideramos la noción de que las flechas signifiquen algo más que herencia. En el diagrama de Priest, significan que $\delta(x)$ trasciende x y que pertenece a Ω. En nuestro caso, podríamos tener dos tipos de aristas: por un lado, las de herencia y, por otro, las de trascendencia y clausura. Con esto, nos referimos a que las aristas puedan cargar un peso proposicional tales como "δ aplicado sobre cierto nodo pertenece a otro cierto nodo". Algo parecido a $\langle \delta, x, \Omega, \in, 0 \rangle$ pero

condensado en una arista.

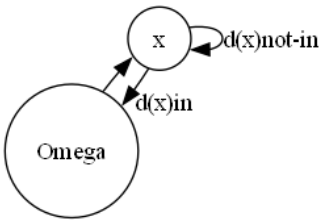
Trabajaremos sobre el *pipeline* de la librería Hypersets quitándolo de la teoría y colocándolo en un entorno híbrido. Esperamos que el cambio de panorama nos brinde ideas para modelar sistemas intrincados y sus propiedades lógicas. Los sistemas de ecuaciones pasarán a ser sistemas de ecuaciones y de instancias del tipo `Function`. Este tipo de datos nos da pie para referirnos a las propiedades que cumplen los elementos de la imagen de la función en cuestión, mediante otro tipo `SetProp`. Para los propósitos de este trabajo, solo modelamos las propiedades de pertenencia ($\in$) y no pertenencia ($\notin$). Sabremos que una variable del sistema de ecuaciones pertenece a la imagen de una función cuando el constructor `Pred` se encuentra en el `SetOf` asociado a la indeterminada en cuestión.

En este caso, la función `schemeToSystem` toma un elemento `Inclosure` y devuelve el sistema de proposiciones asociado. Como podemos ver, no incorporamos los predicados ϕ ni ψ al esquema. Esto se debe a que buscamos que el modelo sea simple y, en todo caso, una base sobre la cual añadir aplicación de funciones booleanas. Eso implicaría ampliar el tipo `FunProp`.

```haskell
data Inclosure t = Inclosure
    { omega :: String -- conjunto "total"
    , x :: String         -- conjunto x
    , label :: String   -- Nombre del diagonalizador
    }
    deriving (Show)

schemeToSystem :: Inclosure String -> System String
schemeToSystem (Inclosure omega x delta) =
    [ Equation omega (SetOf [Ref x]) -- Omega = {x}
    , Equation x (SetOf [Pred delta])
    , Function delta (SetProp [
          Not (Elem (Ref x) (Ref x)), -- delta(x) not-in x
          Elem (Ref x) (Ref omega)] ) -- delta(x) in Omega
    ]
```

A su vez, `denoteSystem` funciona con la misma base que antes y almacena las propiedades que cumple cada valor de la imagen de una función. Para ello, el tipo `RefHFS` pasa a llamarse `RefHFScheme`, que agrega el constructor `Application Name ID [(t, ID, Value)]`. Al encontrarse con `Function`, `denoteSystem` recorre la lista `SetProp` y crea un elemento `Application` con el nombre de la función y el `ID` de la variable a la cual se le aplica, el `Label` y `ID` del conjunto con el que se relaciona, y un `Value` indicando el valor de verdad del predicado de pertenencia. Si en `SetProp` encontramos dos proposiciones contradictorias, `denoteSystem` marca ese caso como una contradicción mediante el `Value` V2.



(a) Diagrama del *Inclosure Scheme* en su totalidad

(b) Diagrama del *Inclosure Scheme* en su caso borde, marcado por $x = \Omega$

Figura 19: Resultados de los experimentos realizados sobre el modelo global del *Inclosure Scheme*

Retomando la distinción entre *shallow* y *deep embedding*, la evaluación de proposiciones y su detección de contradicciones es un caso de *shallow embedding*. La función central para esta tarea es `isContradiction`. A través de *pattern matching* es que se define una inconsistencia lógica como la coexistencia de proposiciones opuestas. No se requieron tipos de datos extras, solo recorrer una lista y detectar la proposición del tipo $p \wedge \neg p$.

```
isContradiction :: FunProp String -> FunProp String -> Bool
isContradiction p (Not q) = p == q
isContradiction (Not q) p = p == q
isContradiction _ _     = False
```

Una vez que se construye el conjunto, no solo con información de herencia de indeterminadas sino también de la imagen de una función, pasamos a la creación de un grafo que recopile esos datos y los represente. La relación de herencia entre conjuntos se expresa de la misma manera, mientras que la relación de pertenecia de $f(x)$ sobre un conjunto y se expresa con una arista saliente del nodo x . Esta arista se dirige hacia el nodo y. La característica distintiva de las aristas de las aplicaciones de funciones y sus propiedades es que se encuentran etiquetadas. Su etiqueta marca la propiedad – $\in$, $\notin$ o $\in$! – que cumple $f(x)$.

Un grafo continúa siendo un arreglo de un vértice v y una lista de vértices asociados. Las aristas son establecidas entre v y cada vértice de la lista. Dado que las aristas pueden estar o no etiquetadas, las etiquetas de las aristas son de tipo `Maybe`. La función `setToLabGraph`, en este caso, usa las ocurrencias de `RefS` para crear aristas simples y las ocurrencias de `Application` para crear aristas etiquetadas. En este último caso, se usa directamente la etiqueta dentro de `Application`. La etiqueta de cada arista o la ausencia de ella representa, respectivamente, la propiedad que cumple la aplicación de una función f o una simple pertenencia de conjuntos.

```
type Table a = Array Vertex a
type Graph e = Table [(Maybe e, Vertex)]
```

El *pipeline* descripto genera la Figura 19a al partir de `schemeToSystem`. Cabe destacar que las etiquetas no contienen símbolos matemáticos, ya que su correcta representación requeriría un manejo explícito de la codificación UTF-8 en los archivos generados.

Ahora bien, la Figura 19b se genera al instanciar x con Ω. Para ello, recurrimos a un sistema de ecuaciones solo con la variable Ω, generado por `schemeBorderCase`. Cuando aplicamos sobre Ω la propiedad de Trascendencia y Clausura sobre sí mismo sucede que, por dentro, el código detecta la contradicción. A partir de ahí, estas dos proposiciones son tratadas como una sola entidad. Es interesante notar que el caso borde tiene la misma forma que G_Ω.

5. Análisis de resultados

Recordemos que uno de nuestros objetivos era crear modelos que nos permitan estudiar las paradojas en sí mismas, ni reducirlas a contradicciones ni abstraerlas en definiciones planas, predecibles y calculadas. En este capítulo, analizaremos el alcance de los modelos y propondremos posibles extensiones.

Por un lado, tenemos los modelos que extienden la teoría de *hypersets* con nuevas estructuras y funciones. Consideramos que son una representación novedosa de la estructura composicional de las paradojas. Gracias a la teoría de *hypersets*, su noción de solución y la estrategia de canonización de grafos, logramos exhibir a las paradojas como un sistema de componentes funcionales, complejos y dinámicos donde la circularidad forma parte intrínseca.

Podemos considerar las n-uplas como el fruto de una representación que coloca la herencia en un lugar fundamental. En su estructura se resalta la referencia y la negación como bases. Por ejemplo, una oración simple como "Esta oración no puede ser escrita en 9 palabras" puede ser modelada como la Figura 18. Allí, vemos las flechas que conectan a cada elemento, la división en subgrupos relacionados y la distinción de ciertos elementos cruciales. Estas características dan un salto estructural a la hora de expresar referencias reflexivas, o sea, referencias a otra expresión del mismo tipo. Podemos apreciar esto último en la Figura 17.

El enfoque adoptado permite recuperar y reorganizar la estructura de las oraciones paradójicas propuestas por Grim et. al y Priest, reinterpretándolas dentro de una teoría de conjuntos no bien fundados. Los resultados mantienen una clara proximidad estructural con sus diagramas, si bien difieren de ellos al inscribirse en un marco explícitamente circular y sin operaciones lógicas de por medio.

Por otro lado, tenemos los modelos del *Inclosure Scheme*. El primero que fabricamos consiste en una serie de n-uplas. Cada una de ellas, nuevamente, es una forma ejemplar de representar estructuralmente cada punto del *Scheme*. En este punto, notamos que expresar el límite contradictorio que define al diagonalizador es esencial a la representación fiel de la propuesta matemática del *Scheme*. Pero, como ya dijimos en el capítulo anterior, solo pudimos definir Clausura y Trascendencia de maneras aisladas, sin dar cuenta de la contradicción $\delta(\Omega) \in \Omega!$ ni mostrar información sobre la injerencia que tiene δ en todo el sistema que buscamos modelar. Aunque apostamos por la representación de la estructura global de las paradojas de autorreferencia, nuestros modelos tienen como límite la expresión de la oscilación entre estados contradictorios y pasos intermedios de inferencia lógica.

Con esta preocupación, encaramos el modelo del *Inclosure Scheme*, siendo una oportunidad perfecta para explorar la estructura enrerada de las paradojas con otro nivel de profundidad. Modelamos la aplicación de predicados sobre Hypersets, cambiando los sistemas de ecuaciones válidos para referirnos a funciones y sus propiedades, pero manteniendo la existencia de conjuntos no bien fundados. Puntualmente, añadimos valores de verdad, propiedades lógicas y detección de contradicciones. Notar que sin la definición de tipos de datos que definan si la aplicación de una función $f(x)$ pertenece a y, no sería posible definir el *pattern matching* para comparar dos predicados de este tipo.

Aunque dimos un paso hacia la inferencia lógica en nuestro grafos, no representamos la oscilación de manera explícita sino que sucede en la construcción interna del grafo. Esto no quita el acierto de incorporar predicados sobre las variables y definiciones de pasos lógicos a través de *shallow embeddings*. Consideramos que los resultados, presentes en la Figura 19, logran transformar la estructura plana del *Inclosure Scheme* al eco circular de la contradicción creada por δ. No es casualidad que el caso borde del *Scheme* sea idéntico al grafo G_Ω. Este es un modelos que, en un futuro, podría generalizarse mediante el uso de una sintáxis formal y emplearse para modelar las paradojas del *Inclosure Scheme*, como las paradojas de Russell, Burali-Forti, Mentiroso, etc..

En este respecto, podría ampliarse la expresividad del modelo logrado con *hypersets* mediante la incorporación de una sintáxis básica. Contar con conjuntivos lógicos, constantes y predicados sería de gran ayuda para representar la oscilación entre veracidad y falsedad, inherente a las paradojas y su contradicción. En líneas generales, el desarrollo de esta idea requiere incorporar herramientas sintácticas para describir de manera precisa las relaciones entre proposiciones del tipo `SelfRefSentence` y `Sentence`. Esto no solo nos daría lugar para crear una representación visual de paradojas sino también nos permitiría representar sus consecuencias. Por ejemplo, podríamos generar un grafo con los pasos lógicos que se encuentran en la demostración del Teorema de Incompletitud de Gödel, de

modo similar a como lo encaran Grim y Rescher.

Otras posibles mejoras para la librería Hypersets consisten en explorar las aplicaciones de la teoría de *hypersets* propuestas por Barwise y Moss. Entre ellas, sugerimos dos como caminos posibles de extensión: lógica modal (Barwise & Moss, 1996, p. 131) y semántica situacional (Barwise & Moss, 1996, p. 177). Estas extensiones podrían brindar un modelo que habite contradicciones de una manera más robusta que nuestra propuesta anterior. En lo que respecta a semántica situacional, Barwise et. al usan las mismas n-uplas que vimos, para construir modelos reflexivos de la paradoja del Mentiroso, y le suman una sintáxis. Además de este último camino, que resulta bastante intuitivo y complementario para nuestros modelos, un enfoque con lógica modal y autómatas de transiciones etiquetadas (Barwise & Moss, 1996, p. 276) podría ser efectivo para habitar la faceta lógica sin dejar de lado lo relacionalmente circular que nos brindan los *hypersets*.

6. Conclusión

En este trabajo nos encargamos de estudiar las estructuras internas que revelan las paradojas lógicas cuando son modeladas computacionalmente como procesos operacionales dentro de la teoría **ZFA**. Nuestro motivo recae, por un lado, en la curiosidad imperante acerca de los fenómenos reflexivos. Por otro lado, es de nuestro gran interés explorar el uso de la computadora como espacio experimental y como herramienta para la construcción de conocimiento filosófico y lógico.

El análisis de antecedentes teóricos sobre la composición y caracterización de las paradojas, con especial atención a los fenómenos de autorreferencia y circularidad, no solo permitió identificar ejemplos paradigmáticos, sino también delimitar un espacio conceptual adecuado para el estudio sistemático de la autorreferencia. A partir de la investigación de los autores, fue posible establecer los elementos indispensables de nuestros modelos estructurales dentro de **ZFA**.

A raíz de esto, decidimos desarrollar una librería computacional basada en la teoría de conjuntos **ZFA**, integrando la explicación teórica con la implementación técnica de sus propiedades fundamentales para simular y visualizar estructuras reflexivas. En el capítulo 3, modelamos el Axioma de Fundación en su forma de grafo y de sistema de ecuaciones. El resultado es un *pipeline* que permite ingresar un sistema de ecuaciones y generar el grafo de herencia, llamado visualización por su característica de mostrar la estructura de pertenencias que hace al conjunto que actúa como etiqueta del nodo raíz. Evaluamos formas de reducir el costo de las funciones así como de extender las funcionalidades actuales.

En este punto, alcanzamos una introducción a *hypersets* y la creación de un entorno oportuno para la realización de experimentos reflexivos. En el capítulo 4, hicimos uso de esta herramienta con el objetivo de explorar y comparar estruc-

turas de un grupo representativo de paradojas. Concretamente, empleamos pares ordenados para modelar, de manera simbólica, los componentes de proposiciones lógicas: un predicado, un valor de verdad y una proposición a modo de sujeto.

Como dijimos en el capítulo 5, el despliegue de la referencia en los grafos muestra la circularidad como parte de la estructura interna de las paradojas. Justamente, las simulaciones expusieron la dimensión referencial de las paradojas de Yablo, del Mentiroso Dual y de proposiciones que se niegan a sí mismas. Para las proposiciones del *Inclosure Scheme*, identificamos los mismos componentes esenciales pero los expusimos de una forma más abstracta y compacta, lo que involucró aristas etiquetadas. En el caso particular de la paradoja de Yablo, que no contiene autorreferencia, se ilustra el problema de la escalera ilimitada de referencias.

A su vez, el enfoque de desarrollo partiendo desde las bases hacia instancias más abstractas resultó crucial para construir modelos con estructura referencial dentro de *hypersets* y dejar espacio para extender la teoría original. Esto permite aplicar la librería a otros proyectos y/o añadir una herramientas lógicas a nuestros modelos de paradojas. Es importante continuar con la investigación en este sentido, para poder modelar la naturaleza de las paradojas en su totalidad y no solo un lado del diálogo: sin inferencias lógicas no podemos simular la dinámica oscilatoria de las paradojas.

Como línea de trabajo futuro, estas extensiones lógicas podrían abordarse de distintas maneras. Una forma posible es, como ya dijimos en el capítulo 5, definiendo un lenguaje con su debida sintáxis, semántica y algunas reglas de inferencia entre proposiciones. También, podría usarse la librería Hypersets como base para la creación de modelos reflexivos mediante la semántica situacional o la incorporación de diagramas de transición etiquetados a partir de lógica modal. De cualquier modo, queda pendiente el trabajo sobre aspectos vinculados a la implementación computacional, tales como criterios de eficiencia y corrección del código.

La investigación presentada abre un conjunto de interrogantes que trascienden el análisis puntual de las paradojas lógicas. En particular, queda abierta la pregunta de hasta qué punto el enfoque basado en *hypersets* puede generalizarse al modelado de otros sistemas reflexivos, tales como demostraciones de teoremas limitantes de la computación y en problemas filosóficos como la conciencia o el libre albedrío[27]. Asimismo, resulta relevante explorar en qué medida el modelado computacional de paradojas mediante *hypersets* puede servir como base para el desarrollo de herramientas pedagógicas orientadas a la enseñanza de Lógica y la Filosofía, especialmente a partir del uso de simulaciones y visualizaciones como mediadores conceptuales.

[27] https://goertzel.org/consciousness/consciousness_paper.pdf

En suma, los grafos y diagramas construidos bajo la teoría de *hypersets*, que fomenta la representación de estructuras cíclicas y referenciales, son un resultado alentador para futuras investigaciones sobre el modelado de la referencialidad en estructuras reflexivas, por sí mismas y buscando un hueco en nuestro conocimiento de ellas. Por lo tanto, el aporte central de este Trabajo Especial de grado es haber construido una herramienta para el laboratorio filosófico – la librería Hyperset – que permite desglosar las paradojas en sus componentes funcionales, tratando la contradicción no como un error, sino como un objeto de estudio dinámico. Esperamos que los experimentos realizados inspiren al lector a ver la computadora como un laboratorio filosófico, a crear modelos lógicos para sus propias investigaciones y a compartir con el mundo los tan fascinantes ciclos extraños.

Anexo

A. Repositorio de código

El código utilizado en este trabajo se encuentra en el repositorio de GitHub: https://github.com/rocio-perez-sbarato/strangelooplab.

Contenido del repositorio

El repositorio contiene un archivo `README.md` con instrucciones detalladas para compilar y ejecutar todos los componentes. A su vez, los componentes principales del repositorio son los siguientes:

1. `hypersets`: la librería Hypersets
2. `hyperset-paradox`: los modelos de proposiciones autorreferenciales y referenciales como extensión de Hypersets.
3. `hyperset-inc-scheme`: el modelo del *Inclosure Scheme* como modificación de Hypersets.

Sugerencias

- Ejecutar cada paquete siguiendo las instrucciones del repositorio. Estos reproducen automáticamente los experimentos completos de los Capítulos 3 y 4.
- Consultar la documentación Haddock del código siguiendo las instrucciones del repositorio.
- Reutilizar o extender `Hypersets` para futuros trabajos en el área.

Bibliografía

Aczel, P. (1988). *Non-well-founded sets*. CLSI Publications. https://doi.org/10.2307/2274782

Barendregt, H. (2003). Reflection and its Use: From Science to Meditation.

Bartlett, S. J. (Ed.). (1992). *Reflexivity: A Source-Book in Self-Reference*. Springer.

Bartlett, S. J., & Suber, P. (Eds.). (1987). *Self-Reference: Reflections on Reflexivity*. Springer.

Barwise, J., & Etchemendy, J. (1987). *The Liar: An Essay on Truth and Circularity*. Oxford University Press, USA. https://doi.org/10.2307/2275189

Barwise, J., & Moss, L. (1996). *Vicious Circles: On the Mathemathics of Non-Wellfounded Phenomena*. CLSI Publications. https://doi.org/10.2307/2275587

Bolander, T. (2002). Self-reference and Logic. *Phi News*, *1*.

Bolander, T., Hendricks, V., & Pedersen, S. (2006). *Self-reference*. Center for the Study of Language; Information.

Boscaratto, S., Nascimben, F., Policriti, A., et al. (2025). Hyperset Individualisation Algorithms. *CEUR WORKSHOP PROCEEDINGS*, *4039*, 67-80. https://hdl.handle.net/11390/1314324.

Boscaratto, S., Omodeo, E., Policriti, A., et al. (2024). On generalised Ackermann encodings–the basis issue. *CEUR WORKSHOP PROCEEDINGS*, *3733*, 1-16. https://hdl.handle.net/11368/3085218.

Data.Graph [Documentation for the `containers` library, version 0.7-8423]. (2025). %5Curl%7Bhttps://downloads.haskell.org/ghc/latest/docs/libraries/containers-0.7-8423/Data-Graph.html%7D

Davis, M. (2018). *The Universal Computer: The Road from Leibniz to Turing*. Taylor & Francis Group. https://doi.org/10.2307/2695463

Devlin, K. (1993). Non-Well-Founded Sets. En *The Joy of Sets: Fundamentals of Contemporary Set Theory* (2.ª ed.). Springer. https://doi.org/10.1007/978-1-4612-0903-4

Dovier, A. Logic programming and bisimulation. En: En *CEUR Workshop Proceedings*. *1433*. CEUR-WS, 2015, 1-14. http://ceur-ws.org/

Gibbons, J., & Wu, N. (2014). Folding domain-specific languages: deep and shallow embeddings (functional Pearl). *49*(9), 339-347. https://doi.org/10.1145/2692915.2628138

Gottwald, S. (2022). Many-Valued Logic. En E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Summer 2022). Metaphysics Research Lab, Stanford University.

Grim, P., Mar, G. R., & Denis, P. S. (1998). *The philosophical computer: Exploratory Essays in Philosophical Computer Modeling*. The MIT Press. https://doi.org/10.2307/3071082

Grim, P., & Rescher, N. (2012). *Reflexivity: From Paradox to Consciousness*. Ontos Verlag. https://doi.org/10.1515/9783110320183

HaskellWiki. (2025). The Monad.Reader/Issue5/Practical Graph Handling [[Online; accessed 21 November 2025]]. %5Curl%7Bhttps://wiki.haskell.org/index.php?title=The_Monad.Reader/Issue5/Practical_Graph_Handling&oldid=67198%7D

Hofstadter, D. R. (1999). *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books. https://doi.org/10.2307/2025598

Lim, D. (2023). *Philosophy Through Computer Science: An Introduction*. Routledge. https://doi.org/10.4324/9781003271284

Morton, T. (2013). *Realist Magic: Objects, Ontology, Causality*. Open Humanities Press. https://doi.org/10.3998/ohp.13106496.0001.001

Omodeo E. G., T. A. I., Policriti A. (2017). Sets, Graphs, and Set Universes. En *On Sets and Graphs: Perspectives on Logic and Combinatorics* (2.ª ed.). Springer. https://doi.org/10.1007/978-3-319-54981-1

Pakkan, M., & Akman, V. (1995). HYPERSOLVER: A graphical tool for commonsense set theory. *Information Sciences*, *85*(1), 43-61. https://doi.org/https://doi.org/10.1016/0020-0255(94)00114-Q

Priest, G. (1995). *Beyond the limits of thought*. Cambridge University Press. https://doi.org/10.1017/S0031819100041528

Primiero, G. (2020). On the Foundations of Computing. Oxford University Press. https://doi.org/10.1093/oso/9780198835646.001.0001

Reynolds, J. C. (1978). User-Defined Types and Procedural Data Structures as Complementary Approaches to Data Abstraction. En D. Gries (Ed.), *Programming Methodology: A Collection of Articles by Members of IFIP WG2.3* (pp. 309-317). Springer New York. https://doi.org/10.1007/978-1-4612-6315-9_22

Tarau, P. (2008). *A Functional Hitchhiker's Guide to Hereditarily Finite Sets, Ackermann Encodings and Pairing Functions*. University of North Texas. [Unpublished draft]. https://doi.org/10.48550/arXiv.0808.0754